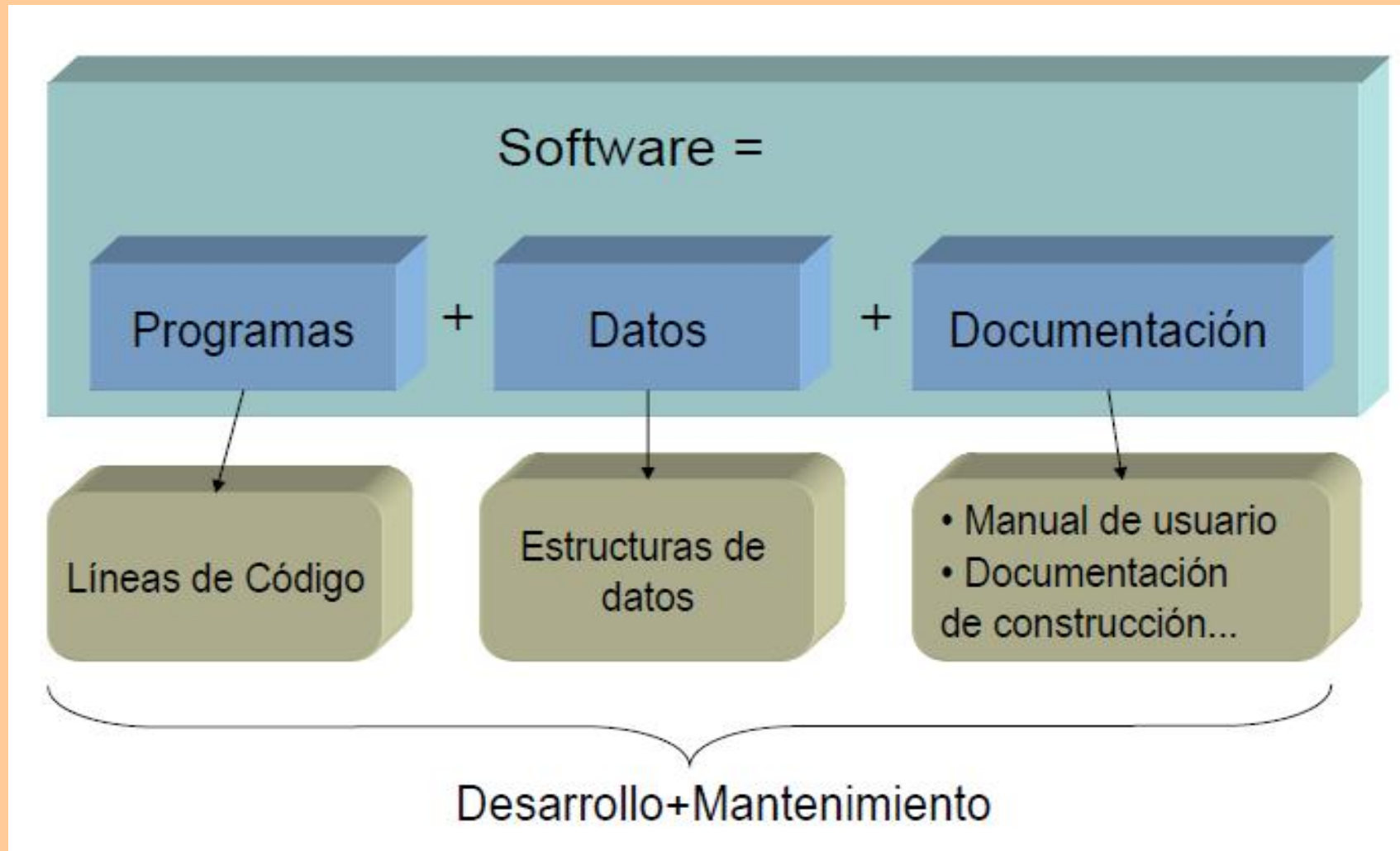


Tema 2: Ingeniería del Software y Gestión de Proyectos

Ingeniería del Software. Características del software. Proceso, modelos, costos y métodos de la Ingeniería del Software. Características y atributos del software. La complejidad inherente al software. Gestión de Proyectos. Métricas de proyectos. Métricas orientadas al tamaño. Métricas orientadas a la función. Eficiencia y calidad. Estimación y planificación de proyectos. Técnicas de estimación, de descomposición y empíricas. Estimación y planificación ágil. Costos y recursos. Técnicas de evaluación y revisión de actividades. Calendarización del proyecto. Análisis y gestión de riesgos. Gestión de calidad. Herramientas CASE-I de planificación y de estimación de proyectos

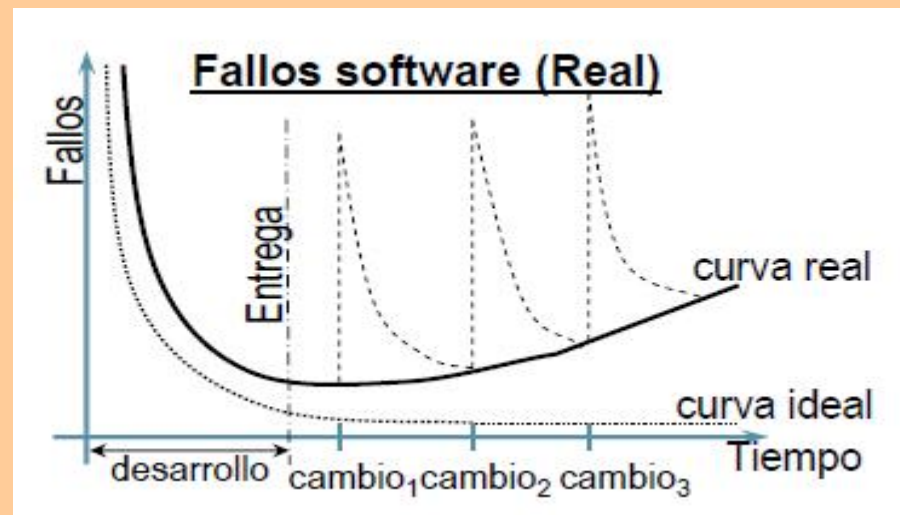
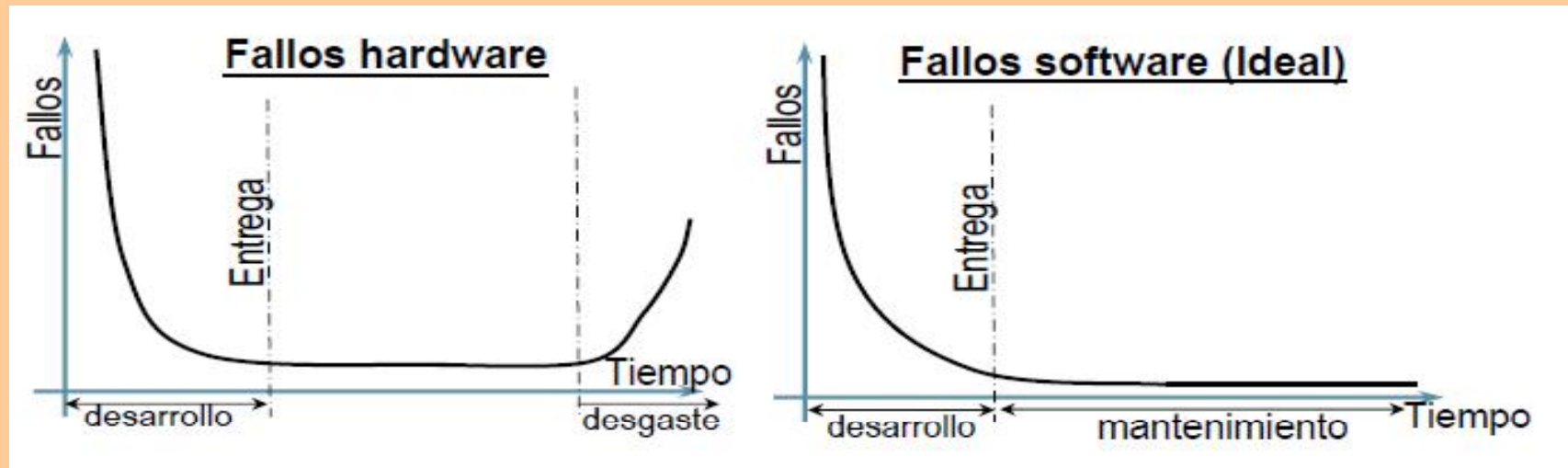
Software: Definición Software



Software: Características Informática

- ☐ Es un elemento lógico y no físico.
- ☐ Es desarrollado, no fabricado.
- ☐ Se deteriora ?
- ☐ No hay “*piezas de repuesto*”.
- ☐ Se construye a medida.

Software: Características



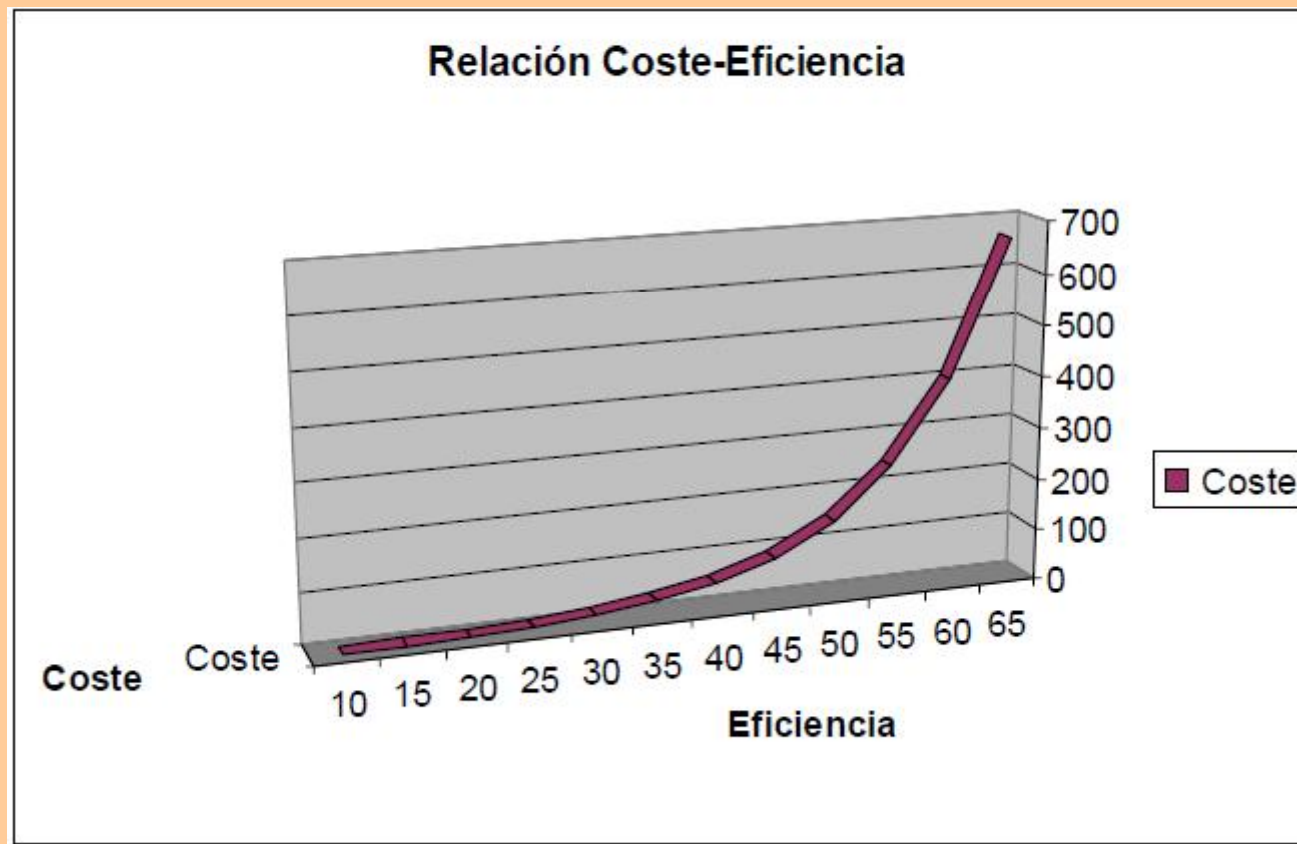
Software: Características

Un sistema software debe ser...

- ☐ Fácil de mantener. Construido y documentado para permitir cambios.
- ☐ Fiable. Debe hacer lo que se espera de él, no debe fallar más a menudo de lo que se acordó en la especificación.
- ☐ Eficiente. No debe hacer uso innecesario de recursos.
- ☐ Fácil de usar. Interfaz de usuario adecuada.

Software: Características Informática

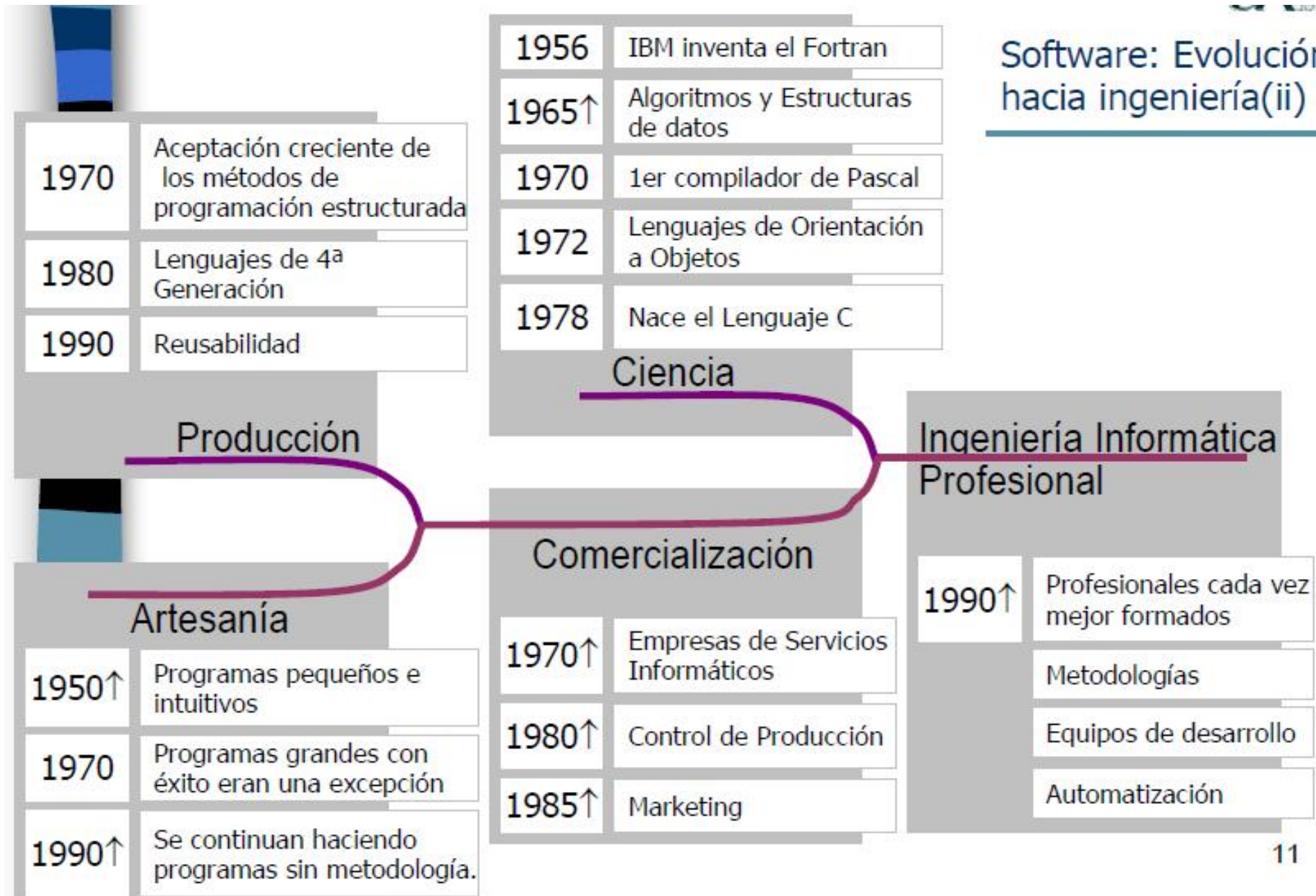
- Algunos de estos atributos son exclusivos.



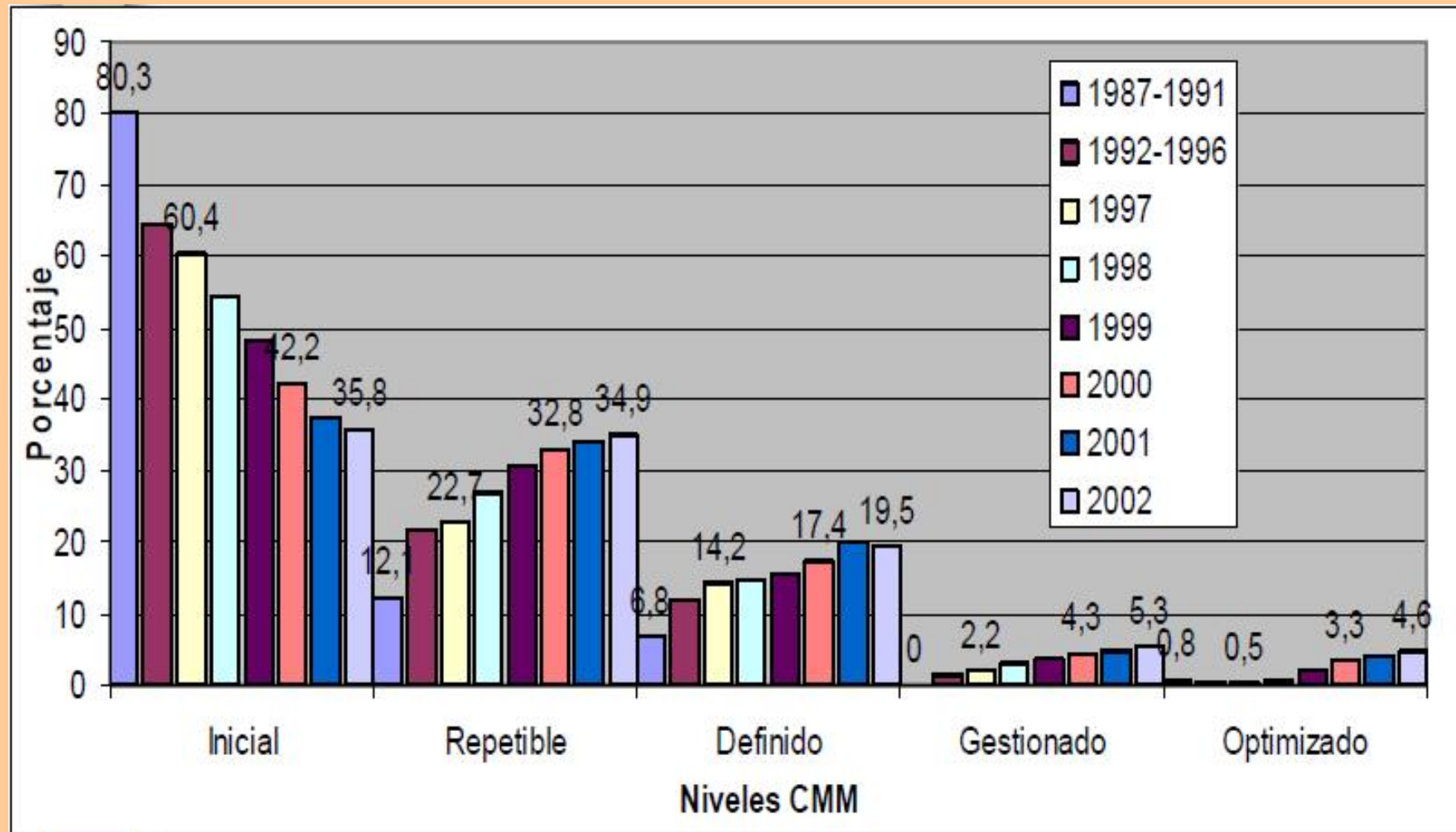
Informática Software: Evolución hacia ingeniería



Software: Evolución hacia ingeniería(ii)



Informática Software: Evolución hacia ingeniería



Software: Progreso de técnicas en su construcción Informática

Programación de cualquier modo



Programación a pequeña escala



Programación a gran escala

	1950-65	1965-75	1975-
Problemas típicos	Programas pequeños	Algoritmos y programación estructurada	Interfaces, Herramientas, Tiempo real, Distribución.
Datos	Información simbólica	Estructuras y tipos de Datos	Bases de Datos
Control	Diagramas elementales de flujo	Programas que se ejecutan 1 vez y terminan	Programas ejecutándose continuamente
Gestión	Ninguno	Individual	Equipo de desarrollo Mantenimiento Garantía de Calidad Métricas
Herramientas y Métodos	Ensamblador	Lenguajes de Programación. Compiladores	Documentación. 4GL CASE

Software: Crisis del Software



- Causas.
 - Hardware más potente.
 - Mayor demanda.
 - Falta de metodologías y técnicas (hasta hace poco).
 - Uso inadecuado de recursos.
 - Sistemas más complejos.
 - Poca información de los desarrolladores.
- Síntomas.
 - Productividad de los desarrolladores: baja en relación a la demanda. Expectativas: los sistemas no responden a las expectativas de los usuarios. Fiabilidad: Los programas fallan a menudo.
 - Calidad: No es adecuada.
 - Costes: Difíciles de predecir, a menudo sobrepasan lo esperado.
 - Mantenimiento: Modificación del sw costosa y compleja.
 - Plazos: No se cumplen.
 - Portabilidad: Difícil cambiar de plataforma.
 - Eficiencia: No hay aprovechamiento óptimo de recursos.
- Consecuencias.
 - Baja productividad.
 - Baja calidad.
- Solución.
 - Aplicar la Ingeniería del Software en la construcción de Sistemas Informáticos.
 - La necesidad de un enfoque de ingeniería en el desarrollo del software fue propuesta en una conferencia de la OTAN en 1968.

Ingeniería del Software: Desafíos

- ❑ Reducir el coste y mejorar la calidad del software.
- ❑ Explotar y aprovechar el potencial proporcionado por el hardware
- ❑ Desarrollar y mantener software asegurando:
 - Calidad.
 - Fiabilidad.
 - Facilidad de uso
 - Imposibilidad de mal uso

...de tal manera que el humano dirija el ordenador y no al revés.

Ingeniería del Software: Desafíos

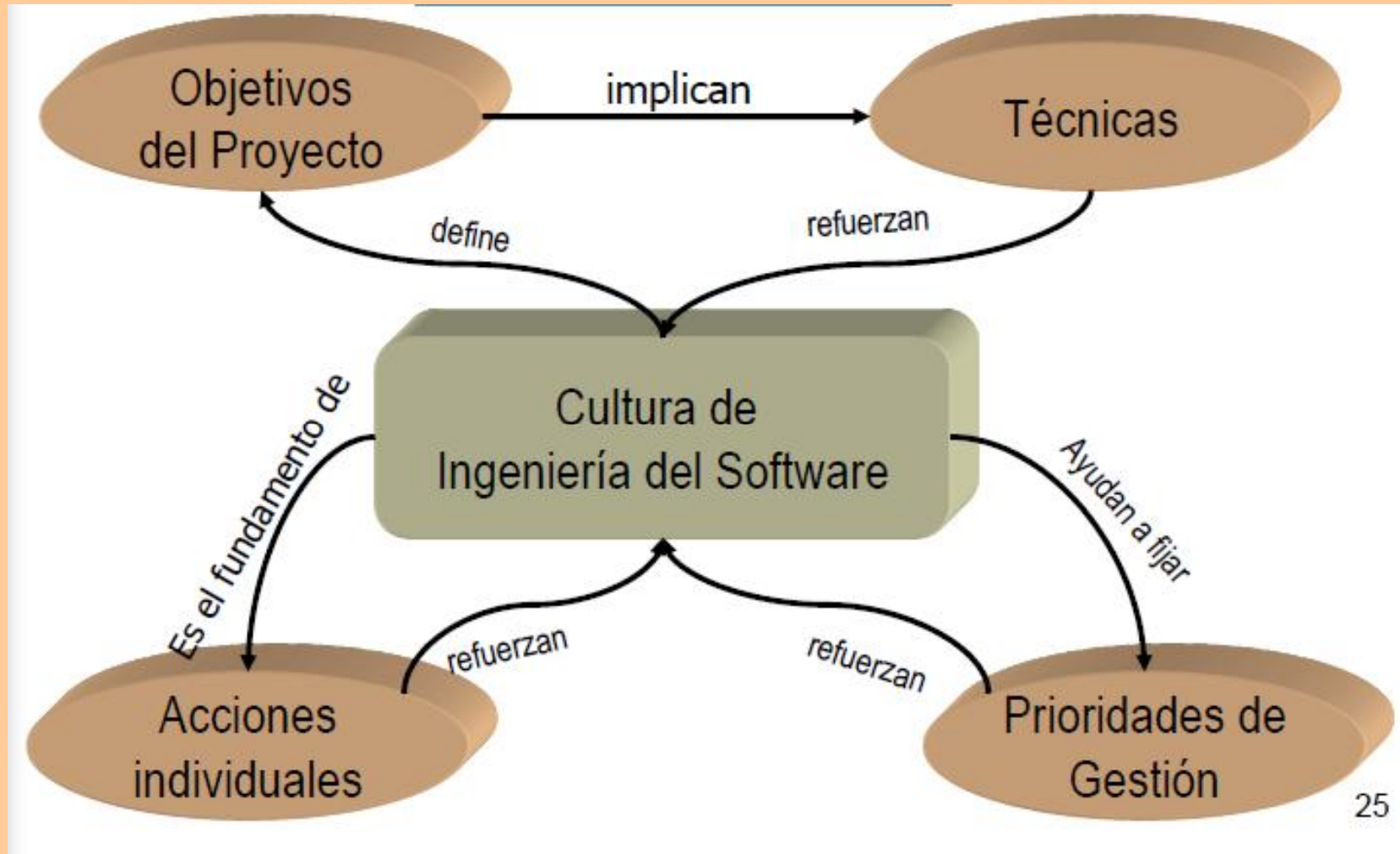
Existen muchas clases de software:

- De sistemas.
- De tiempo real.
- De gestión.
- Científico.
- De Inteligencia Artificial.
- Empotrado.
- Para PC.
- ...

...pero existen métodos y procedimientos comunes para construir buen software
Informática



Ingeniería del Software: Desafíos



Ingeniería del Software: Definición

“El establecimiento y uso de principios de ingeniería robustos, orientados a obtener económicamente software que sea fiable y funcione eficientemente sobre máquinas reales”

Fritz Bauer

“La aproximación sistemática al desarrollo, operación y mantenimiento del software.”

“Software: programas de ordenador, procedimientos, reglas, documentación y datos asociados a un sistema de ordenador”.

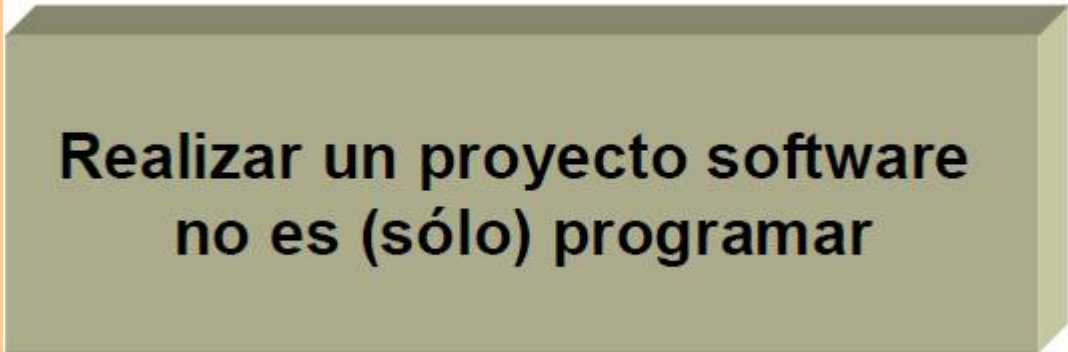
IEEE Standard Glossary of Software Engineering (IEEE83).

Ingeniería del Software: Objetivos

El principal (común a todas las ingenierías)

“Construir instrumentos (HW, SW) que ayuden o faciliten al ser humano la realización de alguna tarea”

- Conseguir un producto Software fiable, de alta calidad y bajo coste.
- Conducir un proceso de desarrollo y mantenimiento software de manera eficiente y con éxito.



**Realizar un proyecto software
no es (sólo) programar**

Ingeniería del Software: Objetivos

Actuar sobre las causas de la crisis del software:

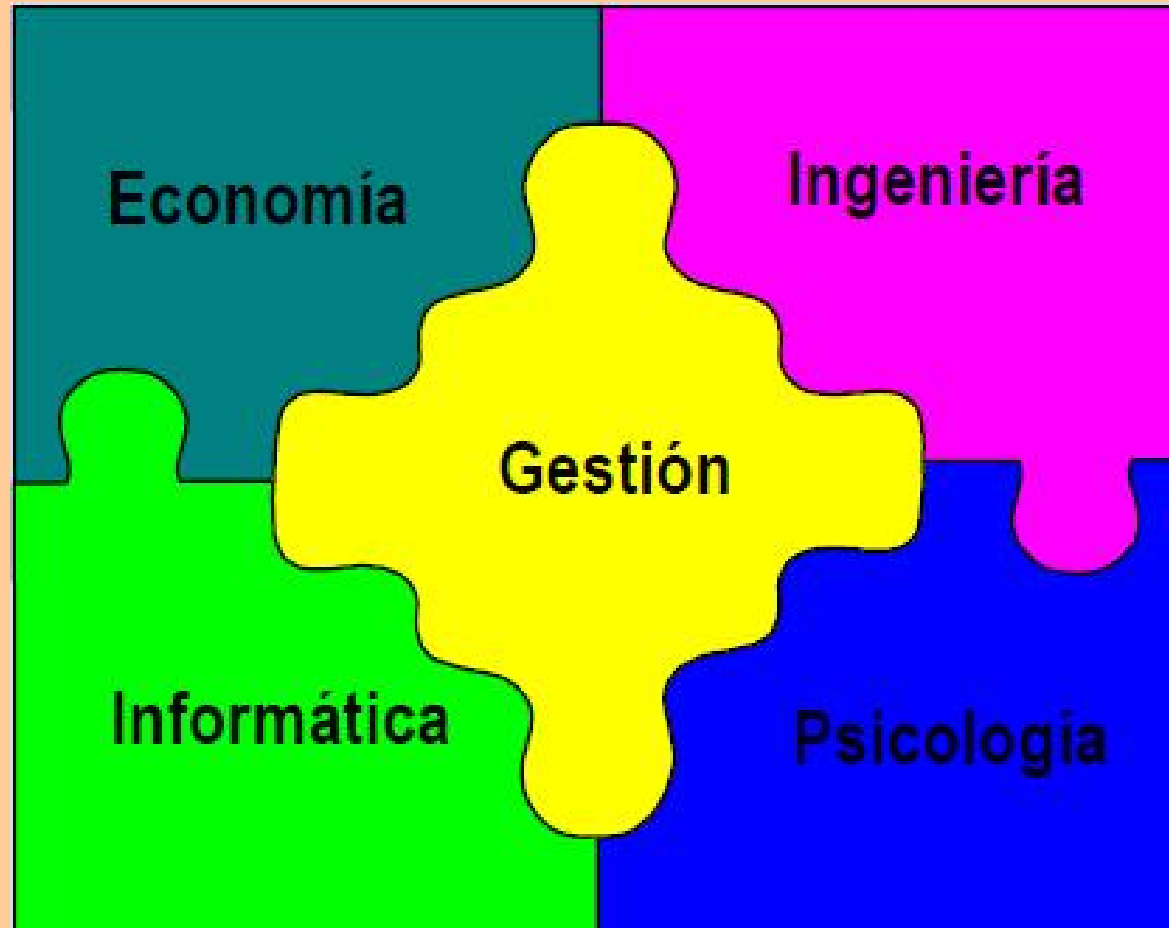
- ☐ Mejorar la calidad de los productos software.
- ☐ Aumentar la productividad de los ingenieros del software.
- ☐ Facilitar el control y seguimiento del proceso de desarrollo.
- ☐ Suministrar a los desarrolladores las bases para construir software de alta calidad de forma eficiente.
- ☐ Definir una disciplina que garantice la producción sistemática y el mantenimiento de los productos software desarrollados en el plazo fijado y dentro del coste estimado.

Ingeniería del Software: Diferencias con otras Ingenierías

Ingenierías: Construyen instrumentos que imitan, aumentan, ayudan, facilitan o sustituyen capacidades físicas del ser humano.

Ingeniería del Software: Construyen instrumentos que imitan, aumentan, ayudan, facilitan o sustituyen capacidades psíquicas del ser humano (imitan y ayudan al ser humano en su capacidad de resolución de problemas).

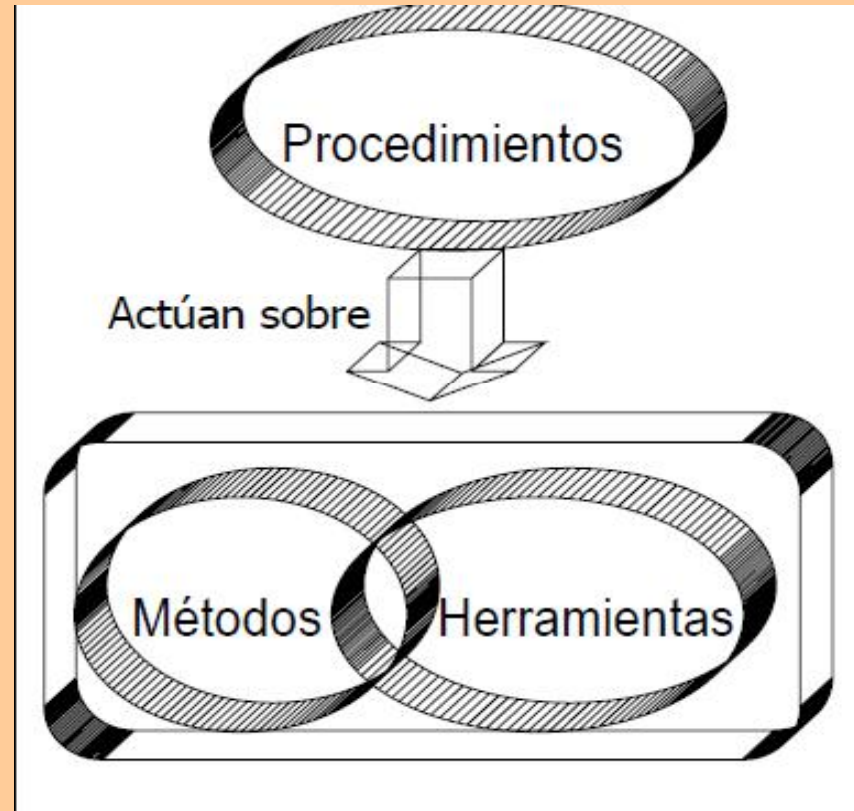
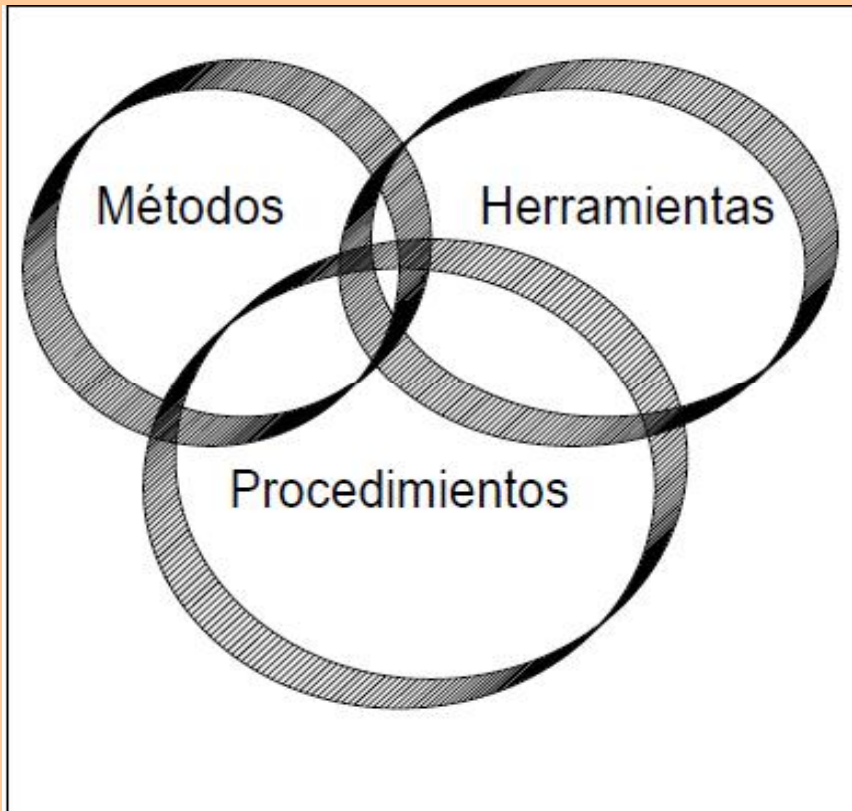
Disciplinas que la componen



Fundamentos y técnicas

- ❑ Métodos. Definen cómo construir el software desde el punto de vista técnico. (Planificación y estimación de proyectos, Análisis de requisitos, Diseño, Codificación, Pruebas, Mantenimiento).
- ❑ Herramientas: Proporcionan un soporte automático o semi-automático para los métodos (CASE, CAD, etc.)
- ❑ Procedimientos. Punto de unión entre métodos y herramientas. Definen la secuencia en la que se aplican los métodos, cómo usar las herramientas, las entregas que se requieren, controles de seguimiento y calidad, guías para facilitar la labor de gestores y desarrolladores, etc.

Fundamentos y técnicas



Ingeniería del Software: Otras técnicas de apoyo

Abstracciones:

- Modelos de ciclo de vida.
- Principios de las distintas fases, ...

Representaciones:

- Notaciones y lenguajes.
- Diagramas de GANTT
- Lenguaje Z,...

Evaluaciones:

- Mediciones (sobre proceso y producto).

Informática Ingeniería del Software:

Actividades del Ingeniero del SW

El Ingeniero del Software debe:

Trabajar en equipo.

Analizar y estudiar problemas.

Trabajar bajo restricciones de tiempo, costes y recursos.

Interactuar con clientes y usuarios.

Tomar decisiones.

...

Actividades del Ingeniero del SW

Tipo de actividades:

- ☐ de desarrollo.
- ☐ de control.
- ☐ de gestión.
- ☐ de operación.

Actividades del Ingeniero del SW

Actividades de desarrollo:

- | | |
|--|-----------------------|
| ☐ Decidir qué hacer. | (Análisis) |
| ☐ Decidir cómo hacerlo. | (Diseño) |
| ☐ Hacerlo. | (Codificación) |
| ☐ Probar el producto. | (Pruebas) |
| ☐ Usar el producto. | (Entrega/Instalación) |
| ☐ Mantener el producto. | (Mantenimiento) |



Actividades del Ingeniero del SW

Actividades de Control: se ocupan de evaluar y asegurar la calidad del software.

- ☐ Métricas.
- ☐ Garantía de calidad.
- ☐ Gestión de configuraciones.



Actividades del Ingeniero del SW

Actividades de Gestión:

- ☐ Planificación y estimación.
- ☐ Seguimiento de los proyectos.
- ☐ Administración de proyectos.
- ☐ Dirección de proyectos.



Actividades del Ingeniero del SW

Actividades de Operación:

- ☐ Entrega (e instalación).
- ☐ Puesta en marcha.
- ☐ Formación a los usuarios

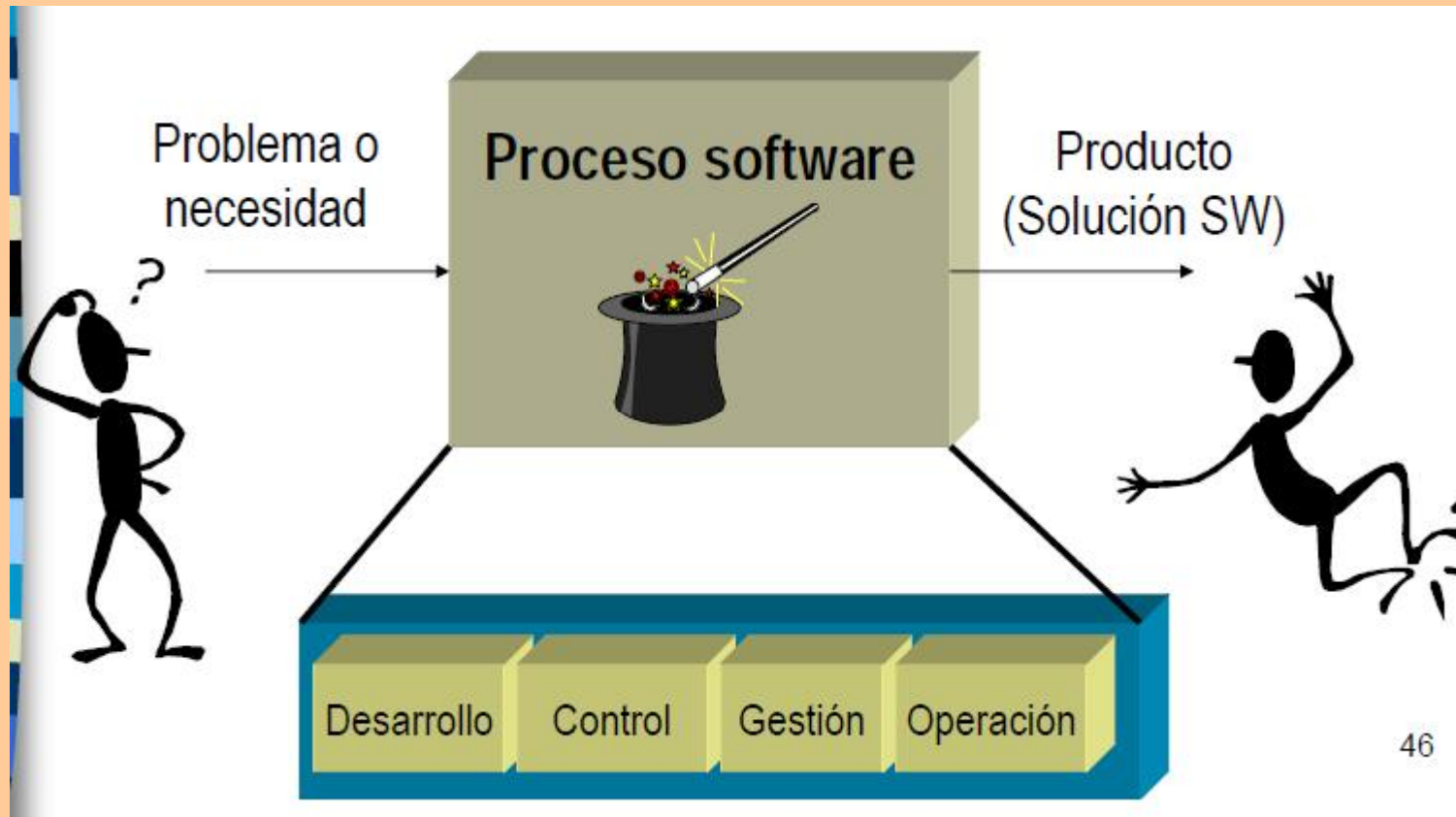


Actividades del Ingeniero del SW

**Realizar un proyecto software
no es (sólo) programar**

**El proceso de construir software
es una actividad de resolución
de problemas**

Producto y Proceso



Producto

Se requieren estimaciones cuantitativas y un plan organizado, pero no se dispone de información sólida.

Un análisis detallado de los requisitos del software proporcionaría la información necesaria para las estimaciones, pero el análisis a menudo lleva semanas o meses.

Los requisitos pueden ser fluidos, cambiando regularmente a medida que progresa el proyecto.

Debemos examinar el producto y el problema a resolver justo al inicio del proyecto. Por lo menos se debe establecer el ámbito del producto y delimitarlo.

Ámbito del software

Contexto. ¿Cómo encaja el software a construir en un sistema, producto o contexto de negocios mayor y qué limitaciones se imponen como resultado del contexto.

Objetivos de información. ¿Qué objetos de datos visibles al cliente se obtienen del software? ¿Qué objetos de datos son requeridos de entrada?.

Función y rendimiento. ¿Qué función realiza el software para transformar la información de entrada en una salida? ¿Hay características de rendimiento especiales que abordar?

El ámbito de un proyecto de software debe ser unívoco y entendible a niveles de gestión y técnico. Los enunciados del ámbito del software deben estar delimitados

Proceso

Las fases genéricas que caracterizan el proceso de software definición, desarrollo y soporte- son aplicables a todo software. El problema es seleccionar el modelo de **proceso apropiado para la ingeniería del software que debe** aplicar el equipo del proyecto.

Paradigmas de ingeniería del software:

- modelo secuencial lineal
- modelo de prototipo
- modelo DRA
- Metodologías Ágiles
 - modelo incremental
 - modelo en espiral
 - modelo en espiral WINWIN
 - modelo de desarrollo basado (ensamblaje) en componentes
 - modelo de desarrollo concurrente
- modelo de métodos formales

Proyecto

Para gestionar un proyecto de software con éxito, debemos comprender qué puede ir mal (para evitar esos problemas) y cómo hacerlo bien.

1. La gente del software no comprende las necesidades de los clientes.
2. El ámbito del producto está definido pobremente.
- 3. Los cambios están mal realizados.**
4. La tecnología elegida cambia.
5. *Las necesidades del negocio cambian [o están mal definidas]*
6. Las fechas de entrega no son realistas.
7. Los usuarios se resisten.
- 8. Se pierden los patrocinadores [o nunca se obtuvieron adecuadamente].**
9. El equipo del proyecto carece del personal con las habilidades apropiadas.
10. Los gestores [y los desarrolladores] evitan buenas prácticas y sabias lecciones.

Proyecto

Se sugiere una aproximación de sentido común a los proyectos de software dividida en cinco partes:

- *Empezar con el pie derecho.*
- *Mantenerse*
- *Seguimiento del Progreso*
- *Tomar Decisiones Inteligentes*
- *Realizar un Análisis «Postmortem» (después definalizar el proyecto).*

Principios para una cultura de IS

- ☐ Nunca dejes que tu jefe o cliente te convenza para hacer un mal trabajo.
- ☐ La gente necesita sentir que su trabajo es apreciado.
- ☐ La educación continua es responsabilidad de cada miembro del equipo.
- ☐ El compromiso del cliente es el factor más crítico en la calidad del software
- ☐ Tu mayor desafío es compartir la visión del producto final con el cliente.
- ☐ La mejora continua de tu proceso de desarrollo de software es posible y esencial.
- ☐ Tener procedimientos escritos de desarrollo de software puede ayudar a crear una cultura compartida de buenas prácticas.
- ☐ La calidad es el principal objetivo; la productividad a largo plazo es una consecuencia de una alta calidad.
- ☐ Haz que los errores los encuentre un colaborador y no un cliente.
- ☐ Una clave en la calidad en el desarrollo de software es realizar iteraciones en todas las fases del desarrollo excepto en la codificación.
- ☐ La gestión de errores y solicitud de cambios es esencial para controlar la calidad y el mantenimiento.
- ☐ Si mides lo que haces, puedes aprender a hacerlo mejor.
- ☐ Haz lo que tenga sentido; no recurras a los dogmas.
- ☐ No puedes cambiar todo de una vez. Identifica los cambios que se traduzcan en los mayores beneficios, y comienza a implementarlos.

GESTION DE PROYECTOS DE SOFTWARE

Mgter. Oscar Adolfo Vallejos – FaCENA - UNNE

- ¿Cómo se debe gestionar el personal, el proceso y el problema durante un proyecto de software?
- ¿Qué son las métricas de software y cómo pueden emplearse para gestionar un proyecto de software y el proceso del software?
- ¿Cómo genera un equipo de software estimaciones fiables del esfuerzo, costes y duración del proyecto?
- ¿Qué técnicas pueden emplearse para evaluar formalmente los riesgos que pueden tener impacto en el éxito del proyecto?
- ¿Cómo selecciona un gestor de proyectos de software el conjunto de tareas del proyecto de ingeniería de software?
- ¿Cómo se crea la planificación temporal de un proyecto?
- ¿Cómo se define la calidad para que pueda ser controlada?
- ¿Qué es la garantía de calidad del software?
- ¿Por qué son tan importantes las revisiones técnicas formales?
- ¿Cómo se gestionan los cambios durante el desarrollo de software de computadora y después de entregarlo al cliente?

Introducción

- Muchos de los problemas en el desarrollo de software: fechas límite, decepción de los clientes, coste disparados, son el resultado de una gestión débil.
- La gestión de proyectos implica la planificación, supervisión y control del personal, del proceso y los eventos que ocurren mientras evoluciona el software desde la fase preliminar a la implementación operacional.
- ¿Quién gestiona?. La respuesta es todos, pero adecuados a su ámbito.
 - Un ingeniero controla las actividades diaria, planificando, supervisando y controlando las tareas técnicas. Los gestores expertos coordinando el negocio y los profesionales del software, etc.

Introducción

El producto que se obtiene en la Gestión es un plan donde se define el proceso, las tareas a realizar, los mecanismos para evaluar los riesgos, mecanismos de seguridad y finalmente controlar el cambio y evaluar la calidad.

¿Cómo estoy seguro que esta correcto? Nunca se esta seguro hasta que se ha entregado el producto. Sin embargo un gestor de proyectos hace lo correcto cuando estimula al personal para trabajar juntos como un equipo efectivo, centrando su atención en las necesidades del cliente y en la calidad del producto.

Componentes de la Gestión

- La importancia de esta tarea de gestión viene dada por la cantidad de tareas y gente que es necesario coordinar en un proyecto software.
- Los pasos en la gestión tienen que cumplir las cuatro Ps → **P**ersonal, **P**roducto, **P**roceso y **P**royecto.
 - Este orden no es arbitrario.
 - Si el gestor olvida que la IS es un esfuerzo humano no tendrá éxito en la gestión de proyectos.
 - Si el gestor no fomenta la comunicación con el cliente construirá un software que no se ajuste a las necesidades.
 - El administrador que se olvida de los procesos no utilizará herramientas que garantizan la calidad del producto.
 - Si el gestor no tiene un plan para el proyecto no alcanzará el producto deseado.

Componentes de la Gestión

Los componentes de la gestión de proyectos son las cuatro Ps: Personal, Producto, Proceso y Proyecto.

Personal: La gestión del personal esta compuesto de varias áreas para garantizar la elección adecuada:

Reclutamiento, Selección, Gestión de Rendimiento Entrenamiento, Retribución, Desarrollo de la Carrera Diseño de la Organización y del trabajo y desarroll cultural y de espíritu de equipo.

Todas estas áreas son denominadas MODELO DE MADUREZ DE GESTIÓN DE PERSONAL.

Producto

Antes de planificar se deben realizar las siguientes tareas:

Objetivos (sin saber como se alcanzarán) y ámbito del producto (dato primarios y funciones y comportamientos que caracterizan al producto); Solucione alternativas; Identificar soluciones técnicas y de gestión. (Estás acciones suelen ser parte del proceso de ingeniería o del negocio).

Proceso

Se establece la estructura desde donde detallar el plan de desarrollo (tareas, hitos, productos, puntos de control para garantía de calidad, etc). Además, en las fases genéricas se selecciona el modelo de proceso (Incremental, DRA, etc).

Modelos CMM/CMMI para la Madurez de la Empresa en la utilización de un proceso.

Proyecto En 1998 los datos de la industria del software indicaron que el 26% de los proyecto fallaron totalmente, del resto el 46% tuvieron desbordamientos en la planificación y el coste. Para evitar el fracaso se debe aplicar el sentido común en la planificación, supervisión y control del proyecto.

Participantes y sus perfiles

En la mayoría de los casos el desarrollo de un sistema de información se lleva a cabo por un grupo de personas. De la armonía que exista entre todos los componentes del grupo de trabajo depende el éxito o fracaso del proyecto.

Atendiendo a las características de complejidad y magnitud del sistema que se esté desarrollando, los perfiles de los participantes que intervienen deberán adaptarse y dimensionarse en cada momento.

Los perfiles establecidos son:

- Directivo.
- Jefe de Proyecto.
- Consultor.
- Analista.
- Programador.

Cada uno de estos perfiles tienen una serie de características importantes a la hora de delimitar su participación en el proyecto.

Perfil de los Directivos

Dentro de esta categoría se agrupan los siguientes participantes:

- Comité Dirección.
- Comité de Seguimiento.
- Directores de Usuarios.
- Usuarios Expertos.

Intervienen en todos los procesos principales, siendo sus principales responsabilidades y funciones similares.

Perfil de los Directivos

El perfil de este grupo de participantes es el siguiente:

- Alto nivel en la dirección de la organización.
- Conocimientos de los objetivos estratégicos y de negocio que se persiguen.
- Autoridad para validar y aprobar cada uno de los procesos realizados durante el desarrollo del sistema.
- Conocimiento del entorno y de la organización suficiente para proporcionar, a lo largo de todo el proyecto, unos requisitos del sistema adecuados, completos y suficientemente importantes como para considerarse en el catálogo definitivo de requisitos.

Comité de Dirección

La responsabilidad del Comité de Dirección es la siguiente:

- o Proveer los recursos necesarios para el cumplimiento de los objetivos propuestos.
- o Revisar y aprobar formalmente cada uno de los procesos.

Comité de Seguimiento y Control

La responsabilidad del Comité de Seguimiento y Control es la siguiente:

- o Resolver cualquier contingencia que pueda presentarse.
- o Asegurar la disponibilidad de los recursos humanos con los perfiles adecuados.
- o Participación en aquellas actividades donde sea necesaria su colaboración.

Directores Usuarios

- La responsabilidad de los Directores de Usuarios es:
 - Aportar información sobre las necesidades planteadas.
 - Validar los resultados con el fin de garantizar la identificación.
 - Comprensión e incorporación de todos los requisitos con las prioridades adecuadas.
- Esta misma función la desempeñan con mayor nivel de detalle los usuarios expertos de nivel directivo.

Perfil del Jefe de Proyecto

Los participantes que se engloban dentro de este perfil son los siguientes:

Jefe de Proyecto.

Responsable de Implantación.

Responsable de Mantenimiento.

Responsable de Operación.

Responsable de Sistemas.

Responsable de Seguridad.

Responsable de Calidad.

Todos ellos ejercen labores de coordinación y dirección de equipos humanos especializados en actividades propias

Jefe de Proyecto

Recibe el apoyo de los distintos responsables durante la realización de procesos o determinadas actividades a lo largo del proyecto.

Tareas:

- Estima el esfuerzo necesario para llevar a cabo el proyecto.

- Selecciona la estrategia de desarrollo.

- Determina la estructura de la estrategia seleccionando los procesos principales.

- Establece la planificación del proyecto, fijando el calendario de hitos y entregas.

- Realiza las labores de seguimiento y control del proyecto.

- Revisa la evaluación de resultados.

- Coordina el equipo de proyecto.

- Gestiona y resuelve las incidencias que puedan surgir durante la realización del proyecto.

- Realiza la actualización de la planificación inicial.

- Realiza los informes de seguimiento.

- Archiva la documentación de gestión del proyecto una vez que éste ha finalizado.

Otros responsables

Los responsables de Implantación

- o Operación, Sistemas y Mantenimiento, participan en los procesos principales del desarrollo, ofreciendo su apoyo al Jefe de Proyecto.
- o Poseen mayor conocimiento de los aspectos organizativos y de procedimientos habituales en la organización en sus áreas de responsabilidad concretas.
- o Aseguran la disponibilidad de los recursos necesarios y la participación activa del equipo humano que coordinan.

Otros Responsables

Los responsables de Seguridad y Calidad

- o Aportan información relativa a las normas y procedimientos habituales en la organización, completándolos en su caso de acuerdo con los requerimientos particulares del sistema en colaboración con el Jefe de Proyecto.
- o Ofrecen asesoramiento sobre todos los aspectos de Seguridad y Calidad relativos tanto al producto como el proceso seguido para su obtención, analizando los riesgos y determinando las medidas de control oportunas.
- o Coordinan a los integrantes de los equipos de Seguridad y Calidad.

Perfil Consultor

Los participantes que se engloban dentro de este perfil son los siguientes:

- Consultor.
- Consultor Informático.
- Especialista en Comunicaciones.
- Técnico de Sistemas.
- Técnico de Comunicaciones.

Perfil Consultor

La función de los consultores es asesorar en las cuestiones sobre las que tienen conocimiento especializado.

Dos tipos de consultores:

- Consultor: asesora en los aspectos relativos al negocio.
- Consultor Informático: asesora en temas informáticos.

En el ámbito de la consultoría informática se distingue entre:

- Tecnologías de la Información.
- Sistemas de Información.

Este consultor aporta:

- Conocimiento de las últimas tecnologías.
- Participa en la evaluación de las distintas alternativas tecnológicas.
- Participa en la validación y selección de la solución más adecuada para el sistema.
- Opinión experta, pericia o conocimientos relativos a los requisitos del negocio, técnicos y de usuario.

Técnicos y especialistas

Los técnicos y especialistas en sistemas y comunicaciones cuentan con una visión más precisa de la tecnología existente en el momento.

Aportan también su experiencia práctica a la hora de valorar alternativas tecnológicas y participan activamente durante su implantación y puesta en producción.

Perfil del Analistas

Los participantes que se engloban dentro de este perfil son los siguientes:

Analista.

Administrador de Bases de Datos.

Equipo de Arquitectura.

Equipo de Formación.

Equipo de Implantación.

Equipo de Operación.

Equipo de Seguridad.

Equipo de Soporte Técnico.

Equipo de Proyecto.

Grupo de Aseguramiento de la Calidad.

Perfil del Analista

Los analistas son los encargados de elaborar un catálogo detallado de requisitos que describan de forma precisa el sistema.

Este catálogo se obtiene mediante entrevistas y sesiones de trabajo con los responsables de la organización y usuarios.

Son los interlocutores entre los usuarios y el equipo de proyecto en lo que se refiere a requisitos.

Con el catálogo de requisitos los analistas elaboran los distintos modelos que sirven de base para el diseño:

- Ej.: Modelo de datos
- Ej.: Modelos de clases e interacción de objetos.
- Ej.: Modelo de Comportamiento
- Ej.: Modelo de Funciones

Administrador de Base de Datos

Participa en las siguientes tareas:

- ✓ Diseño físico de datos, definiendo la estructura física de datos que utilizará el sistema a partir del modelo lógico de datos normalizado o del modelo de clases.
- ✓ Migración de datos, estimando los volúmenes de las estructuras de datos implicadas.
- ✓ Carga inicial de datos.
- ✓ Gestión una vez el sistema está en producción

Equipo de Proyecto

- ✓ Participa a lo largo de todo el proceso de desarrollo y mantenimiento del sistema de información.
- ✓ Su composición puede ir variando en función de las características del proyecto o del proceso que se esté realizando.
- ✓ Se puede diferenciar así los siguientes equipos:
 - Implantación.
 - Operación.
 - Mantenimiento.
 - Arquitectura.
 - Soporte técnico.
 - Seguridad.

Equipo de Formación

El equipo de formación es el encargado de preparar e impartir la formación al equipo responsable de la implantación y operación del sistema. Para ello tiene que elaborar un plan de formación que incluye:

- Cursos de formación y contenidos.
- Recursos humanos.
- Infraestructura.
- Contenido de formación que deberá recibir el usuario.
- Seguimiento de dicha formación.

Grupo Aseguramiento de Calidad

- ✓ Dirigido por el responsable de calidad, este grupo desarrolla el plan de aseguramiento de calidad específico para el proyecto, reflejando las actividades de calidad a realizar.
- ✓ Este grupo participa en la revisión de los productos seleccionados, comprobando que se han llevado a cabo las medidas de calidad establecidas.
- ✓ Este grupo es completamente independiente del equipo del proyecto.

Perfil del Programador

- ✓ Tanto la participación como funciones de los programadores son concretas y limitadas en los procesos de construcción y mantenimiento de los sistemas.
- ✓ Los programadores son miembros del equipo de proyecto, y su función es la de construir el código del producto en base al diseño técnico realizado por el analista-diseñador.
- ✓ El programador es el encargado de realizar las pruebas unitarias del código.

Paradigmas de Organización

La mejor estructura de equipo depende del estilo de gestión de una organización, el número de personas que compondrá el equipo, sus niveles de preparación y la dificultad general del problema.

Existen diferentes organigramas de equipo según diferentes autores, aunque cualquiera que sea la organización del equipo, el objetivo para todos los gestores de proyecto es colaborar para crear un equipo que presente cohesión.

Paradigmas de Organización

A continuación mostramos dos paradigmas:

- o Paradigmas de Mantei (TheEffectofProgramingTea StructuresonProgrammingTask)
 - o Descentralizado Democrático (DD)
 - o Descentralizado Controlado (DC)
 - o Centralizado Controlado (CC).
- o Paradignade Cosntantine (WorkOrganization: ParadigmsforProject ManagementandOrganization).
 - o Cerrado (C)
 - o Aleatorio (AL)
 - o Abierto (A)
 - o Sincronizado (S).

Paradigmas de Mantei

Descentralizado Democrático (DD).

El equipo de ingeniería no tiene un jefe permanente. Se nombran coordinadores de tareas a corto plazo. Las decisiones se toman por consenso. La comunicación del equipo es horizontal.

Los equipos descentralizados producen más y mejores soluciones. La estructura DD es buena para problemas difíciles. Este tipo de organización es para grupos que permanecen mucho tiempo juntos. Aumentan la moral.

Necesitan mucha comunicación. El rendimiento es inversamente proporcional a la cantidad de información. Problemas pequeños. Problemas con baja modularidad. Producen más defectos en los productos. Al ser un equipo descentralizado necesita más tiempo para llevar a cabo una tarea.

Paradigmas de Mantei

Descentralizado Controlado (DC).

Este equipo de ingeniería del software tiene un jefe definido que coordina tareas específicas y jefes secundarios que tienen responsabilidades sobre subtareas.

La resolución de problemas sigue siendo una actividad del grupo, pero la implementación de soluciones se reparte entre los subgrupos por el jefe de equipo.

La comunicación entre subgrupos e individuos es horizontal. También hay comunicación vertical a lo largo de la jerarquía de control.

Buenos para: problemas complejos.
 problemas grandes.
 cuando hay una modularidad alta.

Paradigmas de Mantei

Centralizado Controlado (CC).

El jefe del equipo se encarga de la resolución de problemas a alto nivel y la coordinación interna del equipo. La comunicación entre el jefe y los miembros del equipo es vertical.

Bueno para proyecto grandes. Bueno cuando hay alta modularidad. Buenos cuando producen software similar.

Son menos innovadores.

Paradigma Cosntantine

Cerrado (C).

Paradigma con jerarquía tradicional.

Bueno para proyectos similares.

Poca innovación.

Aleatorio (AL).

Estructura al equipo libremente y depende de la iniciativa individual de los miembros del equipo.

Buenos cuando se requiere innovación y avances tecnológicos.

No son buenos cuando se quiere un rendimiento ordenado.

Abierto (A).

Asigna unos controles al paradigma Aleatorio. El trabajo se desarrolla en colaboración, con mucha comunicación y toma de decisiones consensuadas.

Buenos para problemas complejos.

No son buenos cuando se quiere rendimiento.

Sincronizado (S).

Se basa en la compartimentación natural de un problema y organiza los miembros del equipo para trabajar en partes del problema con poca comunicación activa entre ellos.

Los diferentes paradigmas que se han mostrado anteriormente, pueden ser utilizados de forma híbrida, de la siguiente manera:

Cuando entra alguien nuevo entraría en el modelo CC.

Posteriormente pasaría al modelo DC y finalmente podría pasar al como DD.

Coordinación y Comunicación

Motivos por los que los proyectos de software pueden tener problemas.

Escala; Incertidumbre; Interoperatividad (el software antiguo debe comunicarse con el nuevo).

Para enfrentarse a estos problemas el equipo debe disponer de métodos efectivos para coordinar a la gente, es decir, son necesarios mecanismos de comunicación formales e informales entre todos los miembros (p.e. UML, realización de reuniones).

Coordinación y Comunicación

Formal. Enfoque Impersonal.Incluye documentos de IS y entregas, memorandos técnicos, hitos del proyecto, planificaciones del programa y herramientas de control.

Formal. Procedimientos Interpersonales.Se centra en las actividades de garantía de calidad aplicada a los productos de IS.

Informal. Procedimientos Interpersonales.Reuniones de grupo para la divulgación de información y resolución de problemas.

Comunicación Electrónica.Incluye correo electrónico, boletines de noticias electrónicos y videoconferencias.

Red Interpersonal.Discusiones informales entre los miembros del equipo y con personas que puedan aportar al proyecto aunque no pertenezcan al proyecto.

Reglas para Conseguir un Equipo

Para conseguir un equipo de alto rendimiento:

- Los miembros del equipo deben confiar unos en otros.
- La distribución de habilidades debe adecuarse al problema.
- Para mantener la unión del equipo, los inconformistas tienen que ser excluidos del mismo.
- Cuando se habla de equipo no se habla de grupo de personas, se habla de un grupo cuajado de personas tan fuertemente cohesionado que el todo es mayor que la suma de las partes.

Factores Tóxicos para el Equipo

Atmósfera de trabajo frenética en la que los miembros del equipo gastan energía y se descentran de los objetivos del trabajo a desarrollar.

Alta frustración causada por factores tecnológicos, del negocio, o personales que provocan fricción entre los miembros del equipo.

Procedimientos coordinados pobremente o fragmentados.

Definición confusa de los papeles a desempeñar produciendo una falta de responsabilidad y la acusación correspondiente.

Continua y repetida exposición al fallo, que conduce a la pérdida de confianza y a una caída de la moral.

PROCESO DE SOFTWARE Y MÉTRICAS DE PROYECTOS

Las métricas del software se refieren a un amplio elenco de mediciones para el software de computadora. La medición se puede aplicar al proceso del software con el intento de mejorarlo sobre una base continua. Se puede utilizar en el proyecto del software para ayudar en la estimación, el control de calidad, la evaluación de productividad y el control de proyectos.

Finalmente, el ingeniero de software puede utilizar la medición para ayudar a evaluar la calidad de los resultados de trabajos técnicos y para ayudar en la toma de decisiones táctica a medida que el proyecto evoluciona.

Medidas, métricas e indicadores

Dentro del contexto de la ingeniería del software, una *medida proporciona* una indicación cuantitativa de la extensión, cantidad, dimensiones, capacidad o tamaño de algunos atributos de un proceso o producto.

La *medición* es el acto de determinar una medida.

El *ZEEE Standard Glossary of Software Engineering Terms [IEEE93]* define *métrica* como «una medida cuantitativa del grado en que un sistema, componente o proceso posee un atributo dado».

Una métrica del software relata de alguna forma las medidas individuales sobre algún aspecto.

Un ingeniero del software recopila medidas y desarrolla métricas para obtener indicadores.

Un *indicador* es una métrica o una combinación de métricas que proporcionan una visión profunda del proceso del software, del proyecto de software o del producto en sí

Métricas en el proceso y el dominio del proyecto

Los *indicadores de proceso* permiten a una organización de ingeniería del software tener una visión profunda de la eficacia de un proceso ya existente (por ejemplo: el paradigma, las tareas de ingeniería del software, productos de trabajo e hitos). También permiten que los gestores evalúen lo que funciona y lo que no.

Las métricas del proceso se recopilan de todos los proyectos y durante un largo período de tiempo. Su intento es proporcionar indicadores que lleven a mejoras de los procesos de **software** a largo plazo.

Los *indicadores de proyecto* permiten al gestor de proyectos del software

- (1) evaluar el estado del proyecto en curso;
- (2) seguir la pista de los riesgos potenciales;
- (3) detectar las áreas de problemas antes de que se conviertan en «críticas»;
- (4) ajustar el flujo y las tareas del trabajo,
- (5) evaluar la habilidad del equipo del proyecto** en controlar la calidad de los productos de trabajo del software.

Métricas del proceso y mejoras en el proceso del software

La Única forma racional de mejorar cualquier proceso es medir atributos del proceso, desarrollar un juego de métricas significativas según estos atributos y entonces utilizar las métricas para proporcionar indicadores que conducirán a una estrategia de mejora. Pero antes de estudiar las métricas del software y su impacto en la mejoras de los procesos del software es importante destacar que el proceso es el Único factor de «los controlables al mejorar la calidad del software y su rendimiento como organización»

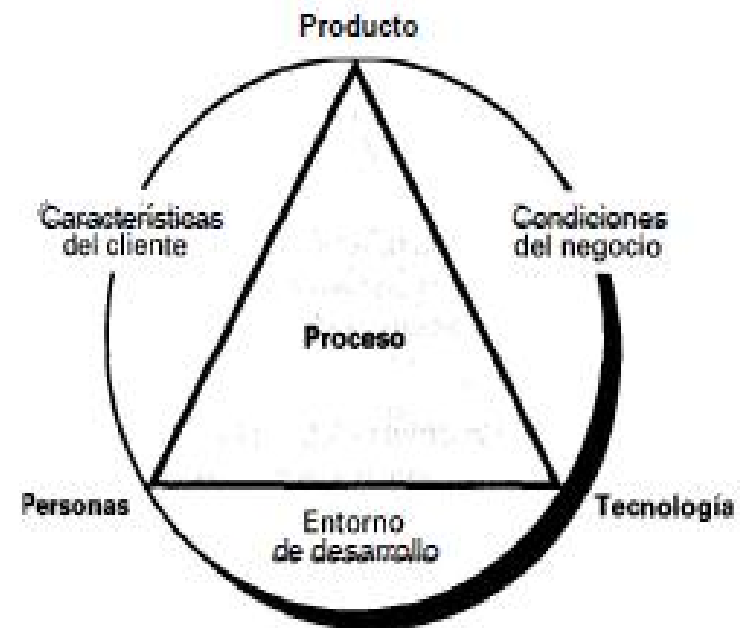


FIGURA 4.1. Determinantes de la calidad del software y de la efectividad de organización (adaptado según [PAU94]).

Métricas del proceso y mejoras en el proceso del software

una «etiqueta de métricas del software» adecuada para gestores al tiempo que instituyen un programa de métricas de proceso:

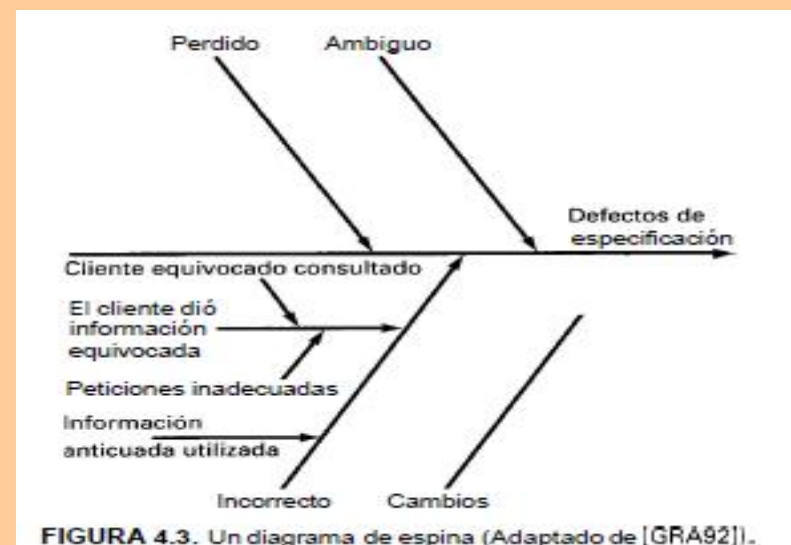
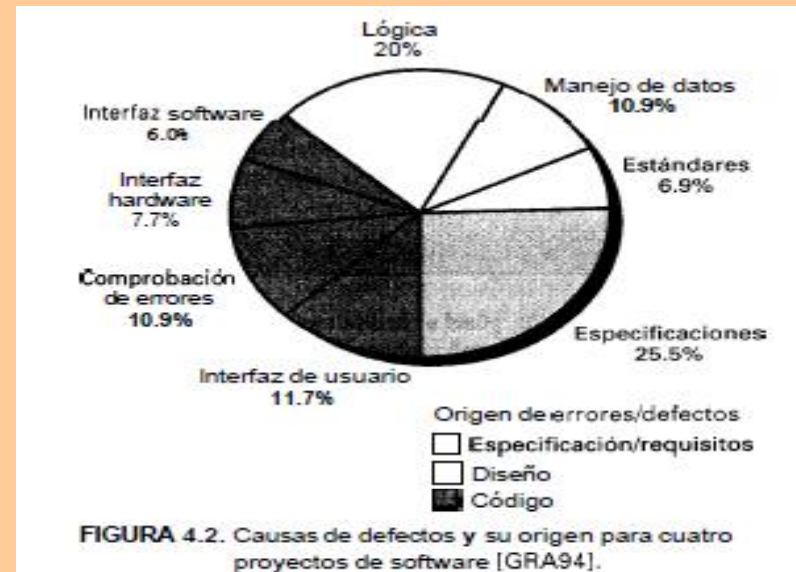
- Utilice el sentido común y una sensibilidad organizativa al interpretar datos de métricas.
- Proporcione una retroalimentación regular para particulares y equipos que hayan trabajado en la recopilación de medidas y métricas.
- No utilice métricas para evaluar a particulares.
- Trabaje con profesionales y equipos para establecer objetivos claros y métricas que se vayan a utilizar para alcanzarlos.
- No utilice nunca métricas que amenacen a particulares o equipos.
- Los datos de métricas que indican un área de problemas no se deberían considerar «negativos». Estos datos son meramente un indicador de mejora de proceso.
- No se obsesione con una sola métrica y excluya otras métricas importantes .

Métricas del proceso y mejoras en el proceso del software

Un enfoque más riguroso llamado *mejora estadística de proceso del software (MEPS)*.

1. **Todos los errores y defectos se categorizan por origen** (por ejemplo: defectos en la especificación, en la lógica, no acorde con los estándares).
2. **Se registra tanto el coste de corregir cada error como el del defecto.**
3. **El número de errores y de defectos de cada categoría** se cuentan y se ordenan en orden descendente.
4. **Se computa el coste global de errores y defectos de cada categoría.**
5. *Los datos resultantes se analizan para detectar las categorías que producen el coste más alto para la organización.*
6. **Se desarrollan planes para modificar el proceso con** el intento de eliminar (o reducir la frecuencia de apariciones de) la clase de errores **y defectos que sean** más costosos.

Mgter. Oscar Adolfo Vallejos – FaCENA - UNNE



Métricas del proyecto

Las métricas del proceso de software se utilizan para propósitos estratégicos. Las medidas del proyecto de software son tácticas (las métricas de proyectos y los indicadores derivados de ellos los utilizan un gestor de proyectos y un equipo de software para adaptar el flujo del trabajo del proyecto y las actividades técnicas).

La primera aplicación de métricas de proyectos en la mayoría de los proyectos de software ocurre durante la estimación. (métricas recopiladas de proyectos anteriores se utilizan como una base desde la que se realizan las estimaciones del esfuerzo y del tiempo para el actual trabajo del software. **A medida que avanza un** proyecto, las medidas del esfuerzo y del tiempo consumido se comparan con las estimaciones originales (y la planificación de proyectos).

El gestor de proyectos utiliza estos datos para supervisar y controlar el avance.

A medida que comienza el trabajo técnico, otras métricas de proyectos comienzan a tener significado. Se miden los índices de producción representados mediante páginas de documentación, las horas de revisión, los puntos de función y las líneas fuente entregadas.

Métricas del proyecto

La utilización de métricas para el proyecto tiene dos aspectos fundamentales.

- ❑ En primer lugar, estas métricas se utilizan para minimizar la planificación de desarrollo haciendo los ajustes necesarios que eviten retrasos y reduzcan problemas y riesgos potenciales.
- ❑ En segundo lugar, las métricas para el proyecto se utilizan para evaluar la calidad de los productos en el momento actual y cuando sea necesario, modificando el enfoque técnico que mejore la calidad.

A medida que mejora la calidad, se minimizan los defectos, y al tiempo que disminuye el número de defectos, la cantidad de trabajo que ha de rehacerse también se reduce. Esto lleva a una reducción del coste global del proyecto.

Métricas del proyecto

Otro modelo de métricas del proyecto de software [HET93] sugiere que todos los proyectos deberían medir:

- ❑ Entradas: la dimensión de los recursos (p. ej.: personas, entorno) que se requieren para realizar el trabajo,
- ❑ Salidas: medidas de las entregas o productos creados durante el proceso de ingeniería del software,
- ❑ Resultados: medidas que indican la efectividad de las entregas.

En realidad, este modelo se puede aplicar tanto al proceso como al proyecto.

Mediciones del software

Las mediciones del mundo físico se pueden categorizar de dos maneras; *medidas directas* (por ejemplo: la longitud de un tomillo) y *medidas indirectas* (por ejemplo: la «calidad» de los tomillos producidos, medidos contando los artículos defectuosos).

Las métricas del software se pueden categorizar de forma similar.

Entre las medidas directas del proceso de la ingeniería del software se incluyen el coste y el esfuerzo aplicados.

Entre las medidas directas del producto se incluyen las líneas de código (LDC) producidas, velocidad de ejecución, tamaño de memoria, y los defectos informados durante un período de tiempo establecido.

Entre las medidas indirectas se incluyen la funcionalidad, calidad, complejidad, eficiencia, fiabilidad, facilidad de mantenimiento y muchas otras «capacidades».

Mediciones del software

- El dominio de las métricas del software se dividen en métricas de proceso, proyecto y producto.
- También se acaba de destacar que las métricas de producto que son privadas para un individuo a menudo se combinan para desarrollar métricas del proyecto que sean **públicas** para un equipo de software.
- Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.

Métricas orientadas al tamaño. Se considera el tamaño del software que se produce.

Métricas orientadas a la función.

Métricas orientadas al tamaño

Para desarrollar métricas que se puedan comparar entre distintos proyectos, se seleccionan las líneas de código como valor de normalización. Con los rudimentarios datos contenidos en la tabla se pueden desarrollar para cada proyecto un conjunto de métricas simples orientadas al tamaño:

- errores por KLDC (miles de líneas de código)
- defectos por KLDC
- E por LDC
- páginas de documentación por KLDC

Además, se pueden calcular otras métricas interesantes:

- errores por persona-mes
- LDC por persona-mes
- **E por página de documentación**

Proyecto	LDC	Esfuerzo	Coste £(000)	Pag. Doc.	Errores	Defectos	Personas
Alfa	12,100	24	168	365	134	29	3
Beta	27,200	62	440	1224	321	86	5
Gamma	20,200	43	314	1050	256	64	6
.	.	.	.	.	.	.	.

FIGURA 4.4. Métricas orientadas al tamaño.

Métricas orientadas a la función.

Las métricas del software orientadas a la función utilizan una medida de la funcionalidad entregada por la aplicación como un valor de normalización. Ya que la funcionalidad no se puede medir directamente, se debe derivar indirectamente mediante otras medidas directas.

Las métricas orientadas a la función fueron propuestas por primera vez por Albretch, quien sugirió una medida llamada *punto de función*.

Número de entradas de usuario. Se cuenta cada entrada de usuario que proporciona diferentes datos orientados a la aplicación. Las entradas se deberían diferenciar de las peticiones, las cuales se cuentan de forma separada.

Número de salidas de usuario. Se cuenta cada salida que proporciona al usuario información orientada a la aplicación. En este contexto la salida se refiere a informes, pantallas, mensajes de error, etc. Los elementos de datos particulares dentro de un informe no se cuentan de forma separada.

Número de peticiones de usuario. Una petición se define como una entrada interactiva que produce la generación de alguna respuesta del software inmediata en forma de salida interactiva. Se cuenta cada petición por separado.

Número de archivos. Se cuenta cada archivo maestro lógico (esto es, un grupo lógico de datos que puede ser una parte de una gran base de datos o un archivo independiente).

Número de interfaces externas. Se cuentan todas las interfaces legibles por la máquina (por ejemplo: archivos de datos de cinta o disco) que se utilizan para transmitir información a otro sistema.

Parámetros de medición	Cuenta	Factor de ponderación				
		Simple	Medio	Complejo		
Número de entradas de usuario		x 3	4	6	=	
Número de salidas de usuario		x 4	5	7	=	
Número de peticiones de usuario		x 3	4	6	=	
Número de archivos		x 7	10	15	=	
Número de interfaces externas		x 5	7	10	=	
Cuenta total						

FIGURA 4.5. Cálculo de puntos de función.

Métricas para la calidad del software

La calidad de un sistema, aplicación o producto es tan bueno como los requisitos que describen el problema, el diseño que modela la solución, el código que conduce a un programa ejecutable, y las pruebas que ejercitan el software para detectar errores.

Un buen ingeniero del software utiliza mediciones que evalúan la calidad del análisis y los modelos de diseño, el código fuente, y los casos de prueba que se han creado al aplicar la ingeniería del software.

Para lograr esta evaluación de la calidad en tiempo real, el ingeniero debe utilizar *medidas técnicas que evalúan* la calidad con objetividad, no con subjetividad.

Visión general de los factores que afectan a la calidad

McCall y Cavano definieron un juego de factores de calidad como los primeros pasos hacia el desarrollo de métricas de la calidad del software.

Estos factores evalúan el software desde tres puntos de vista distintos:

- (1) operación del producto (utilizándolo),
- (2) revisión del producto (cambiándolo), y
- (3) transición del producto (modificándolo para que funcione en un entorno diferente, por ejemplo, «portándolo»).

Medidas de calidad

Corrección. Un programa debe operar correctamente o proporcionará poco valor a sus usuarios. La corrección es el grado en el que el software lleva a cabo su función requerida. (defectos por KLDC).

Facilidad de mantenimiento. Es la facilidad con la que se puede corregir un programa si se encuentra un error, se puede adaptar si su entorno cambia, o mejorar si el cliente desea un cambio de requisitos. No hay forma de medir directamente la facilidad de mantenimiento; por consiguiente, se deben utilizar medidas indirectas. (tiempo medio de cambio (TMC)).

Integridad. Para medir la integridad, se tienen que definir dos atributos adicionales: amenaza y seguridad.

- ☐ Amenaza es la probabilidad (que se puede estimar o deducir de la evidencia empírica) de que un ataque de un tipo determinado ocurra en un tiempo determinado.
- ☐ La seguridad es la probabilidad (que se puede estimar o deducir de la evidencia empírica) de que se pueda repeler el ataque de un tipo determinado.

Facilidad de uso. La facilidad de uso es un intento de cuantificar «lo amigable que puede ser con el usuario» y se puede medir en función de cuatro características:

- (1) habilidad intelectual y/o física requerida para aprender el sistema;
- (2) el tiempo requerido para llegar a ser moderadamente eficiente en el uso del sistema;
- (3) aumento neto en productividad (sobre el enfoque que el sistema reemplaza) medida cuando alguien utiliza el sistema moderadamente y eficientemente;
- (4) valoración subjetiva (a veces obtenida mediante un cuestionario) de la disposición de los usuarios hacia el sistema.

Eficacia de la Eliminación de Defectos (*EED*)

Es una medida de la habilidad de filtrar las actividades de la garantía de calidad y de control al aplicarse a todas las actividades del marco de trabajo del proceso.

Cuando un proyecto se toma en consideración globalmente, EED se define de la forma siguiente:

$$EED = E / (E + D)$$

donde E es el número de errores encontrados antes de la entrega del software al usuario final y D es el número de defectos encontrados después de la entrega.

Integración de las Métricas dentro del proceso de IS

- **Argumentos para las Métricas del Software**
- **Establecimiento de una Línea Base**
- **Colección de métricas, cómputo y evaluación**



Establecimiento de un programa de metrica

La guía sugiere los siguientes pasos para trabajar:

- ☐ Identificar los objetivos del negocio.
- ☐ Identificar lo que se desea saber o aprender.
- ☐ Identificar los subobjetivos.
- ☐ Identificar las entidades y atributos relativos a esos
- ☐ Formalizar los objetivos de la medición.
- ☐ Identificar preguntas que puedan cuantificarse y los indicadores relacionados que se van a usar para ayudar a conseguir los objetivos de medición.
- ☐ Identificar los elementos de datos que se van a recoger para construir los indicadores que ayuden a responder a las preguntas planteadas.
- ☐ Definir las medidas a usar y hacer que estas definiciones sean operativas.
- ☐ Identificar las acciones que serán tomadas para mejorar las medidas indicadas.
- ☐ Preparar un plan para implementar estas medidas.

Estimación y Planificación de Proyectos Software

Planeación y control de actividades

La planeación incluye a todas las actividades que se requieren para la selección del equipo de análisis de sistemas, la asignación de proyectos apropiados, la estimación de tiempo que cada tarea requiere para su ejecución, y la programación del proyecto, de tal forma que las tareas se concluyan oportunamente.

El control denota el uso de la retroalimentación para darle seguimiento al proyecto, esto incluye comparar lo planificado con lo realizado, tomar acciones adecuadas para acelerar o reprogramar las actividades y en consecuencia se concluyan a tiempo y la motivación de los miembros del equipo.

Se debe determinar el grado de detalle que se usara para definir las actividades.

La estimación de duración de cada una de las actividades, para ello podemos usar la experiencia o bien proyectar estimaciones pesimistas y optimistas y obtener un promedio de ambas.

Herramientas: Gantt y PERT

Estimación y Planificación

Estimación = predicción de duración, esfuerzo y costes requeridos para realizar todas las actividades y constituir todos los productos asociados con el proyecto.

La ***planificación es el proceso de:***

- ☐ Selección de una estrategia para la obtención de unos productos finales dados.
- ☐ Definición de actividades a realizar para lograr ese objetivo.
- ☐ Coordinación de concurrencia y solapamiento de dichas actividades.
- ☐ Asignación de recursos a las mismas.

Estimación y Planificación

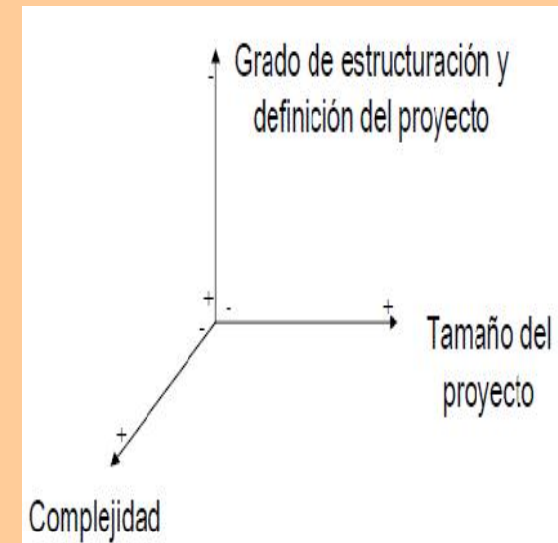
Estimación: Predicción cuantitativa de aspectos del Proyecto.

Planificación: Organización de gente y tareas.

La Planificación es el primer paso en la realización de un proyecto informático y en gran parte, el responsable del éxito o fracaso del mismo.

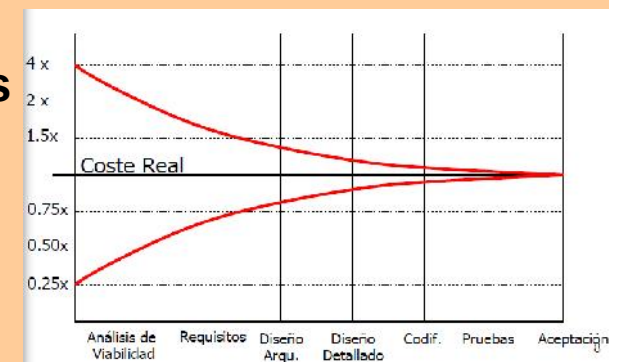
Aspectos que afectan a la estimación

- ❑ *La complejidad del proyecto tiene un gran efecto en la incertidumbre, que es inherente en la planificación.*
- ❑ *El tamaño del proyecto es otro factor importante que puede afectar a la precisión y a la eficiencia de las estimaciones.*
- ❑ *El grado de incertidumbre estructural tiene también efecto en el riesgo de la estimación. En este contexto, la estructura se refiere al grado en el que los requisitos se han definido, la facilidad con la que pueden subdividirse funciones, y la naturaleza jerárquica de la información que debe procesarse.*



El riesgo se mide por el grado de incertidumbre en las estimaciones.

Mgter. Oscar Adolfo Vallejos – FaCENA - UNNE



Elementos sobre los que hay que estimar

Elemento	Unidad
Duración	Tiempo (meses, años, etc)
Esfuerzo	Unidad de esfuerzo (p/m, p/año, etc)
Coste	Euros
Tamaño	Lineas de código (LDC, KLDC) Puntos de Función (PF)

Técnicas de estimación

- ❑ ***Técnicas de descomposición. Descomposición del problema y/o proceso. Estimación del esfuerzo.***
- ❑ ***Técnicas empíricas. Usa fórmulas derivadas empíricamente para predecir el esfuerzo en función de LDC, PF (atributos del software terminado).***

Varios modelos:

COCOMO (I y II).

PUTNAM.

Modelo de Puntos de Función.

- ❑ ***Herramientas automáticas de estimación.***

Técnicas de descomposición: Estimación del esfuerzo

- Enfoque “divide y vencerás”.
- Descomponer en módulos, tareas o funciones.
- Modelo matricial.
- Modelo jerárquico.

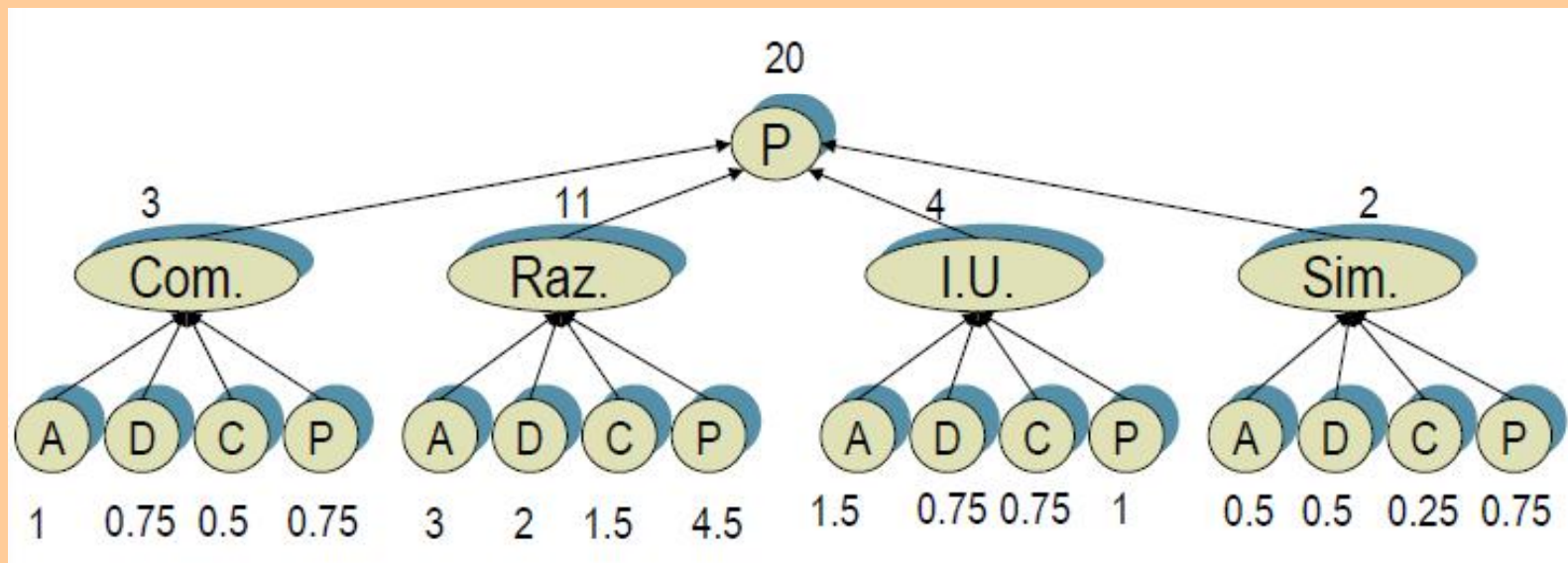
Modelo Matricial

- ❑ Descomponer en funciones/tareas.
- ❑ Realizar una matriz.
- ❑ Estimar el esfuerzo para cada celda de la matriz.
- ❑ Calcular la suma total.
- ❑ Aplicar costes estándar de la empresa.

	Análisis	Diseño	Codificación	Pruebas	Total (pm)
Comunicaciones	1	0,75	0,5	0,75	3
Razonamiento	3	2	1,5	4,5	11
Interfaz Usuario	1,5	0,75	0,75	1	4
Simulador	0,5	0,5	0,25	0,75	2
Total	6	4	3	7	20
Tarifa (Euros)	7300	6000	5500	5500	
Coste Total	43800	24000	16500	38500	122800

Modelo Jerárquico

- ❑ Descomponer las tareas/módulos en forma de árbol.
- ❑ Calcular el esfuerzo de cada nodo de abajo hacia arriba.



Técnicas empíricas: COCOMO

- CONstructive COst MOdel.
- Propuesto por Barry Boehm en 1981. *“Software Engineering Economics”*.
- *Los datos empíricos se obtienen de una muestra limitada de proyectos.*
- *El modelo no es adecuado para todas las clases de software y en todos los entornos de desarrollo.*
- *Grandes variaciones si no se calibran bien. p.e. Kemerer 1987. 230 pm -3857 pm.*

Versiones del modelo para tres clases distintas de proyectos:

Tipo de Proyecto	Requisitos	Tamaño del proyecto	Complejidad	Nº de personas	Experiencia
Modo orgánico	Poco rígidos	Pequeño (<50KLDC)	Pequeña	Pocas	Mucha
Modo semi-acoplado o semi-rígido	Poco/Medio	Medio [50,300] KLDC	Medio	Medio	Medio
Modo empotrado o rígido	Alto	Grande (>300KLDC)	Alta	Alta	Poca

Técnicas empíricas: COCOMO

Tres tipos de modelos:

- ❑ Básico. Modelo estático. Se da el esfuerzo en función del tamaño del software en líneas de código.
- ❑ Intermedio. Considera más factores (*conductores de coste*) que el tamaño, p.e. *fiabilidad requerida, tamaño de la base de datos, restricciones de ejecución o almacenamiento, etc.*
- ❑ Avanzado o Detallado. Además evalúa el impacto de los conductores de coste en cada fase del proceso de ingeniería del software.

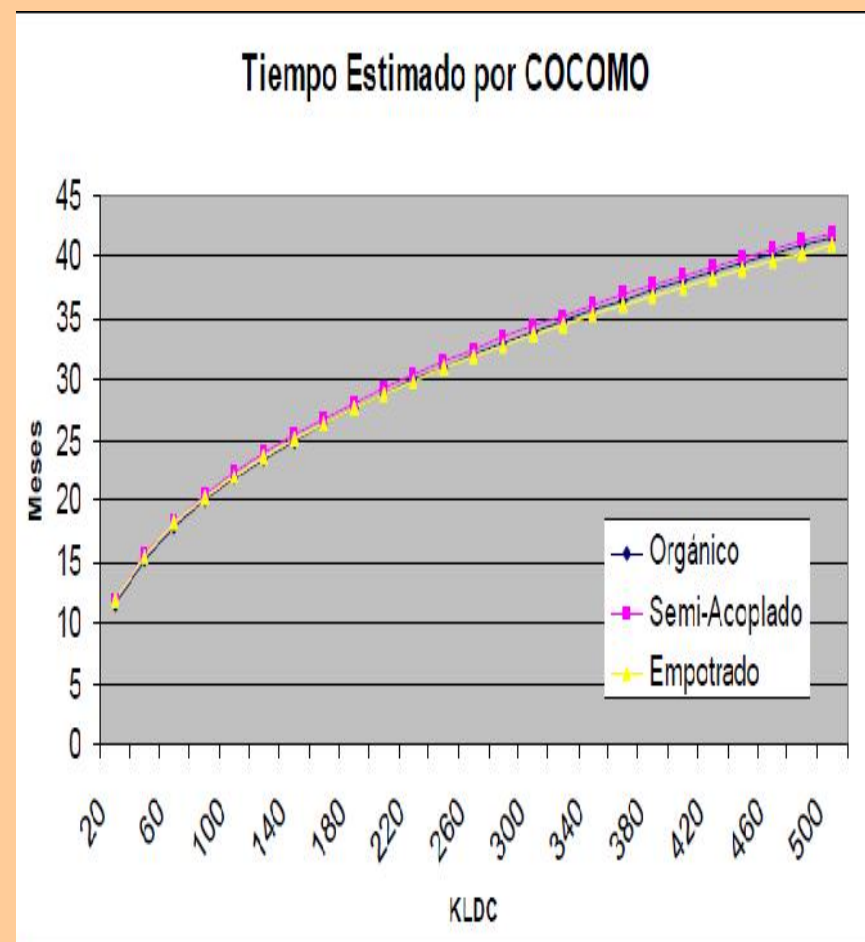
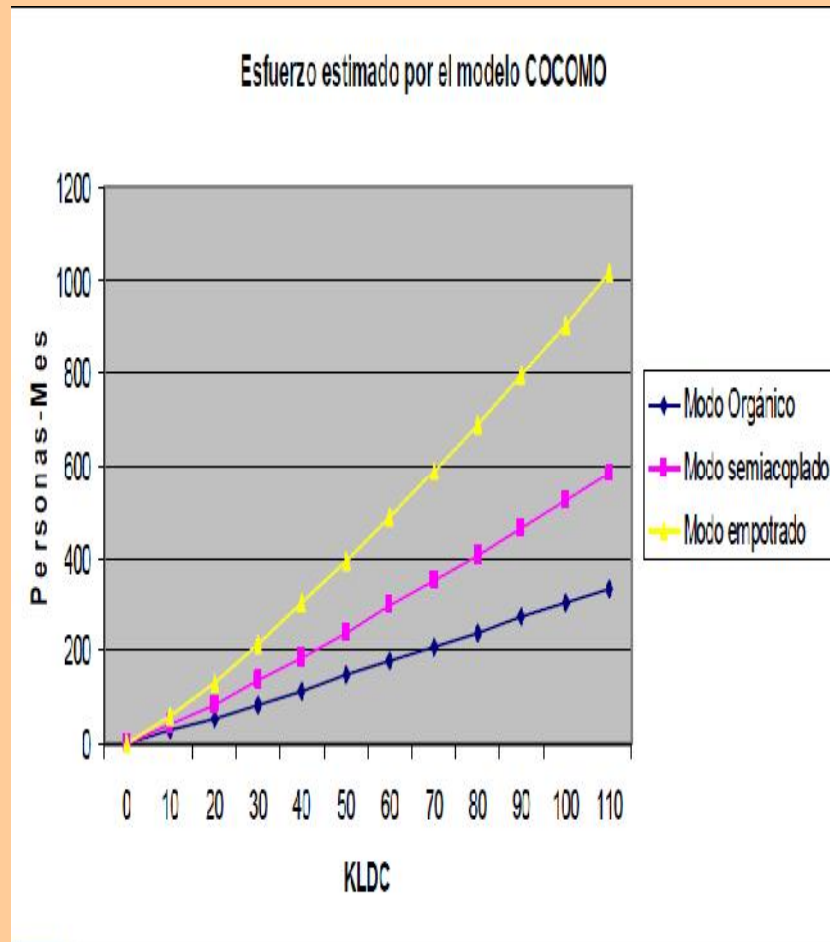
Técnicas empíricas: COCOMO básico

$$E = a \times KLDC^b \quad - \quad D = c \times E^d$$

Donde E es el esfuerzo en personas/mes y D es la duración en meses.
Los coeficientes varían dependiendo del tipo de proyecto.

Tipos de Proyecto	a	b	c	d
Orgánico	2.4	1.05	2.5	0.38
Semiacoplado	3	1.12	2.5	0.35
Empotrado	3.6	1.2	2.5	0.32

Técnicas empíricas: COCOMO básico



Técnicas empíricas: COCOMO básico

La cantidad de gente que se necesita en el proyecto no es uniforme.

Se puede dividir el esfuerzo a lo largo del calendario.

Putnam sugiere las curvas de Rayleigh para este propósito.

$$\text{Manpower}(t) = 2 k a t \exp(-at)$$

Manpower(t): Son las personas/mes de esfuerzo en el mes t.

k : es el esfuerzo total, incluido mantenimiento.

a : es el factor de aceleración.

Varía con el tipo de proyecto y es una constante que se determina empíricamente.

Técnicas empíricas: COCOMO intermedio

Incorpora 15 variables de predicción que influyen en el coste del proyecto.

Seis posibles valores:

- Muy bajo
- Bajo
- Nominal
- Alto
- Muy Alto
- Extra alto.

Un valor bajo implica un multiplicador menor que 1, nominal implica un multiplicador igual a 1, uno alto es mayor que 1.

Técnicas empíricas: COCOMO intermedio

1.- Atributos del producto software.

Fiabilidad requerida del software (RELY).

Fiabilidad= Probabilidad de que el software realice correctamente sus funciones durante un periodo de tiempo dado.

Nivel bajo si un fallo no es muy importante

Nivel nominal si un fallo resulta en pérdidas recuperables.

Nivel muy alto si un fallo puede costar vidas humanas.

Tamaño y complejidad de la base de datos (DATA).

Un valor bajo si el tamaño de la base de datos es menor que 10 veces el tamaño del programa.

Un valor medio si está entre 10 y 100.

Un valor alto si es más de 1000 veces mayor.

Complejidad del producto (CPLX).

Baja si usa operaciones simples de E/S, estructuras de datos simples, etc.

Nominal si tiene procesamiento de E/S, usa varios ficheros, etc.

Muy alta si usa código re-entrante o recursivo, manejo de ficheros complejo, etc.

Técnicas empíricas: COCOMO intermedio

2. Atributos del hardware.

Limitaciones en el tiempo de ejecución (TIME).

Nominal si se usa menos del 50% del tiempo de ejecución disponible.

Muy alto si el 95% del tiempo disponible debe ser usado.

Limitaciones en la memoria principal (STOR).

Nominal si se usa menos de la mitad de la memoria disponible.

Muy alto si se usa más del 95% de la memoria disponible.

Volatilidad de la máquina virtual (VIRT). Máquina virtual= es la combinación de HW y SW.

Bajo si este factor cambia poco (una vez al año).

Nominal si hay muchos cambios(cada 6 meses).

Muy alto si cambia muy frecuentemente.

Tiempo de respuesta (TURN).

Bajo si es un sistema interactivo.

Muy alto, si el tiempo de respuesta es mayor de 12 horas.

Técnicas empíricas: COCOMO intermedio

3. Atributos del personal.

Capacidad de los ingenieros del software senior (ej. analistas)(ACAP).

Baja si menos de tres años.

Alta si más de 6 años.

Capacidad de los ingenieros del software junior (ej. programadores) (PCAP).

Muy baja si poca experiencia.

Media si al menos un año.

Alta si más de tres años.

Experiencia en desarrollo de aplicaciones (AEXP).

Muy baja si menos de cuatro meses.

Muy alta si más de doce años.

Experiencia en el entorno hw/sw de desarrollo (VEXP).

Muy baja si es menos de un mes.

Muy alta si es más de tres años.

Experiencia con el lenguaje de programación (LEXP).

Muy baja si es menos de un mes.

Muy alta si es más de tres años.

Técnicas empíricas: COCOMO intermedio

4. Atributos del proyecto.

Uso de herramientas para desarrollo de software (TOOL). Herramientas CASE, generadores de código, etc.

Aplicación de métodos de Ingeniería del Software (MODP). Análisis estructurado.

Limitaciones en la planificación temporal (SCED). Porcentaje de retraso o aceleración respecto a la planificación impuesta por el equipo de desarrollo.

Técnicas empíricas: COCOMO intermedio

Variable	Muy Bajo	Bajo	Nominal	Alto	Muy Alto	Extra Alto
RELY	0.75	0.88	1.0	1.15	1.4	
DATA		0.94	1.0	1.08	1.16	
CPLX	0.7	0.85	1.0	1.15	1.3	1.65
TIME			1.0	1.11	1.3	1.66
STOR			1.0	1.06	1.21	1.56
VIRT		0.87	1.0	1.15	1.3	
TURN		0.87	1.0	1.07	1.15	
ACAP	1.46	1.19	1.0	0.86	0.71	
AEXP	1.29	1.13	1.0	0.91	0.82	
PCAP	1.42	1.17	1.0	0.86	0.70	
VEXP	1.21	1.10	1.0	0.90		
LEXP	1.14	1.07	1.0	0.95		
MODP	1.24	1.10	1.0	0.91	0.82	
TOOL	1.24	1.10	1.0	0.91	0.83	
SCED	1.23	1.08	1.0	1.04	1.10	

Valores de los conductores de coste

Conductor	Muy Bajo	Bajo	Normal	Alto	Muy Alto	Extra
RELY	Pequeña inconveniencia	Bajo, pérdidas fácilmente recuperables	Moderado, pérdidas recuperables	Pérdidas financieras altas	Riesgos para vidas humanas	
DATA		DB/Prog < 10	10 < D/P < 100	100<D/P< 1000	DB/Prog < 1000	
CPLX						
TIME			<= 50% t. Ejec	<= 70% t. Ejec	<= 85% t. Ejec	<=95% t. Ejec
STOR			<= 50% memo.	<= 70% memo.	<= 85% memo.	<=95%memo.
VIRT		Cambio mayor 12 meses, menor: 1 mes.	Cambio mayor 6 meses, menor: 2 sem.	Cambio mayor 2 meses, menor: 1 sem.	Cambio mayor 2 sem, menor: 2 días.	
TURN		Interactivo	Media < 4 horas	4-12 horas	> 12 horas	
ACAP	15° percentil	35° percentil	55° percentil	75° percentil	90° percentil	
AEXP	< 4 meses	1 año	3 años	6 años	12 años	
PCAP	15° percentil	35° percentil	55° percentil	75° percentil	90° percentil	
VEXP	< 1 mes	4 meses	1 año	3 años		
LEXP	< 1 mes	4 meses	1 año	3 años		
MODP	No uso	Principio de uso	Algún uso	Uso general	Uso rutinario	
TOOL	Herr. Básicas (micro)	Herr. Básicas (mini)	Herr. Básicas (midi/maxi)	Herr. Potentes. De prog, y test	Herr. requ./dis./gest./doc.	
SCED	75% del nominal	85%	100%	130%	160%	

Mgter. Oscar Adolfo Vallejos – FaCENA - UNNE

Técnicas empíricas: COCOMO intermedio

- Se aplica la ecuación de esfuerzo del COCOMO básico.

■ $E \text{ (personas/mes)} = a \times KLDC^b$

Tipos de Proyecto	a	b
Orgánico	3.2	1.05
Semiacoplado	3	1.12
Empotrado	2.8	1.2

■ Calcular el *FAE*, mediante la multiplicación de los 15 valores hallados.

■ El esfuerzo total es:

$$E(\text{persona/mes}) = a \times KLDC^b \times FAE$$

29

Técnicas empíricas: COCOMO detallado

- ❑ Proporciona un conjunto de multiplicadores de esfuerzo para cada atributo en cada fase.
- ❑ Organiza el producto en tres niveles:
 - ❑ Nivel módulo.
 - ❑ Nivel subsistema.
 - ❑ Nivel sistema.

Técnicas empíricas: COCOMO. Críticas

- Se derivó de un conjunto de proyectos ejecutados en los años 60 y 70.
- Hay que ajustar el modelo con métricas de proyectos propios acabados.
- Estimar el esfuerzo requiere conocer el tamaño del software.

Técnicas empíricas: COCOMO II

- ❑ Actualización de las técnicas propuestas por COCOMO para adaptarlo a:
 - ❑ Modelos de ciclo de vida no secuenciales y de prototipado rápido.
 - ❑ Modelos basados en reutilización de productos COTS.
 - ❑ Desarrollo basado en componentes.
 - ❑ Generación de aplicaciones.
 - ❑ Enfoques orientados a objetos y middleware distribuido.

Técnicas empíricas: COCOMO II

- ❑ Utiliza 3 tipos de cantidades de medida para el Software:
 - ❑ Líneas de Código (SLOC)
 - ❑ Puntos de Función sin ajustar (PF).
 - ❑ Puntos de Objeto (PO).

- ❑ Introduce los siguientes modelos de estimación (cada uno soporta distinta granularidad):
 - ❑ Modelo de composición de aplicaciones. Usa PO.
 - ❑ Modelo de Diseño preeliminar. Usa PF + “Backfiring”.
 - ❑ Modelos similares a COCOMO. Usa PF + “Backfiring” o SLOC.
 - ❑ Nuevos modelos para reutilización.

Técnicas empíricas: Modelo de Putnam

$$K = \frac{LDC^3}{C^3 T^4}$$

K = esfuerzo durante todo el ciclo de vida en personas/año.

LDC = número de líneas de código.

T = tiempo de desarrollo en años.

C = constante del estado de la tecnología.

Entorno	C
Pobre	2000
Bueno	8000
Excelente	11000

Técnicas empíricas: Modelo de puntos de función

Se basa en el diseño lógico para cualificar la funcionalidad externa que proporciona el software.

Objetivos:

Proporcionar métricas sobre:

Productividad.

Calidad.

Coste.

Documentación.

Líneas de Código.

Ofrecer un factor de normalización para comparar diferentes software.

Medir lo que el usuario pide y lo que el usuario recibe.

Técnicas empíricas: Modelo de puntos de función

Considera los siguientes parámetros:

Entrada (External Input, EI).

Salida (External Output, EO).

Consultas (External Query, EQ).

Grupos de datos lógicos internos (Internal Logic File, ILF).

Grupos de datos lógicos externos (External Interface File, EIF).

Con estos parámetros se obtienen los Puntos de Función sin Ajustar.

Al valor hallado se le aplica un Factor de Ajuste ($FPA = FP \times AF$).

En base a valoraciones subjetivas sobre la aplicación y su entorno, e.d. características generales del sistema.

Técnicas empíricas: Modelo de puntos de función

Pasos del modelo:

- ☐ Definición de los límites del sistema informático.
- ☐ Definición de parámetros.
- ☐ Valoración de la complejidad para cada parámetro.
- ☐ Análisis de las características generales del sistema para determinar el factor de ajuste.

Técnicas empíricas: Modelo de puntos de función

Paso 1:

Definir el alcance del sistema informático.

Qué se considera parte de la aplicación y qué no.

Técnicas empíricas: Modelo de puntos de función

Paso 2: Definición de parámetros (i)

Componentes que formarán la aplicación final.

Son de dos clases, datos y transacciones:

Tipos de Función de Datos.

Ficheros lógicos Internos. Grupo de datos lógicamente relacionados, utilizados o mantenidos dentro de los límites de la aplicación. Ej.: Base de datos interna, buffers, colas, etc.

Ficheros de Interfaz Externos. Grupo de datos lógicamente relacionados utilizados por la aplicación pero mantenidos por otra aplicación.

Tipos de Función de Transacciones.

Entradas externas. Son datos o información de control que se introduce en la aplicación desde fuera de sus límites. Ej.: Pantallas de entrada, Transacciones en una B.D., etc.

Salidas Externas. Datos o información que sale de los límites de la aplicación. Ej.: informes generados, mensajes de error, informes on-line, transferencia de datos a otras aplicaciones, etc.

Consultas Externas. Entrada interactiva que produce la generación de alguna respuesta del software inmediata en forma de salida interactiva. Ej.: Búsqueda de datos, menús, ayudas, pantallas de conexión, tutoriales, etc.

Paso 3: Valoración de la complejidad

$$\text{Total} = \sum n^{\circ} \text{ parámetros}(\text{dep.complejidad}) \cdot \text{Peso}$$

Parámetro	Complejidad	Peso
Entrada	Alta	6
	Media	4
	Baja	3
Salida	Alta	7
	Media	5
	Baja	4
Fichero Lógico Interno	Alta	15
	Media	10
	Baja	7
Fichero Lógico Externo	Alta	10
	Media	7
	Baja	5
Consultas	Alta	6
	Media	4
	Baja	3

La complejidad de *Entrada*, *Salida* y *Consultas* depende del n° de campos que contengan y del número de ficheros a los que hagan referencia.

La complejidad de *Ficheros* viene dada en función del número de registros y campos que tengan.

Paso 4: Análisis de las características generales del sistema

Valores de ajuste de la complejidad:

1. Comunicación de datos
2. Funciones distribuidas
3. Rendimiento
4. Configuraciones fuertemente utilizadas
5. Frecuencia de transacciones
6. Entrada on-line de datos
7. Diseño para la eficiencia del usuario final
8. Actualización on-line
9. Procesos complejos
10. Utilización en otros sistemas
11. Facilidad de instalación
12. Facilidad de operación
13. Instalación de múltiples sitios
14. Facilidad de cambio

Paso 4: Análisis de las características generales del sistema

A partir de los valores [0,5] asignados a estas características se calcula el *TDI* (*Total Degree of Influence*) y a partir de este el *AF*.

$$AF = (TDI \times 0.01) + 0.65$$

La fórmula final para el cálculo de los puntos de función ajustados es :

$$FPA = FP \times AF$$

Paso 4: Análisis de las características generales del sistema

Comunicación de datos	
La aplicación es por lotes o utilizando un ordenador personal.	0
La aplicación es por lotes pero existe una entrada de datos o impresión remotas.	1
La aplicación es por lotes pero la entrada de datos y la impresión son remotas.	2
Entrada on-line de datos a un sistema de consultas.	3
Más de un ordenador front-end, pero la aplicación soporta sólo un tipo de protocolo de comunicaciones	4
Más de un ordenador front-end y la aplicación soporta más de un tipo de protocolo de comunicaciones	5

Mide cómo llegan los datos a la aplicación

Paso 4: Análisis de las características generales del sistema

Funciones distribuidas	
No existen funciones distribuidas en la aplicación.	0
La aplicación prepara datos que el usuario final procesa en otro componente del sistema.	1
La aplicación prepara datos que son transferidos y procesados por otro componente del sistema, pero no por el usuario final.	2
El proceso distribuido y la transferencia de datos son on-line y en una sola dirección.	3
El proceso distribuido y la transferencia de datos son on-line y en ambas direcciones.	4
Los procesos se desarrollan de forma dinámica en el componente más apropiado del sistema.	5

Mide la existencia de datos o procesos distribuidos dentro del límite de la aplicación.

Paso 4: Análisis de las características generales del sistema

Rendimiento	
No existen requisitos específicos de rendimiento.	0
El rendimiento y requisitos de diseño han sido definidos y revisados pero no requieren ninguna acción especial.	1
El tiempo de respuesta o la capacidad de proceso es crítico durante las horas punta y no se requiere ningún diseño especial para la utilización de la CPU.	2
El tiempo de respuesta o la capacidad de proceso es crítico durante todas las horas de operación, pero no se requiere ningún diseño especial para la utilización de la CPU.	3
Se requiere un diseño especial para tener en cuenta los requisitos de rendimiento ya que éstos son muy estrictos por parte del usuario.	4
Sobre el punto 4, además hay que utilizar herramientas para el análisis de rendimiento durante la fase de diseño, codificación y/o instalación. También hay que verificar los requisitos de rendimiento.	5

Requisitos de usuario tanto en tiempo de respuesta como en volumen de datos a procesar.

Paso 4: Análisis de las características generales del sistema

Configuraciones fuertemente utilizadas	
No existen restricciones de ningún tipo.	0
Existen restricciones operativas, pero no requieren un esfuerzo especial para conseguirlas.	1
Existen algunas restricciones de seguridad o tiempo.	2
Existen requisitos específicos de procesador para algunas partes de la aplicación.	3
Las restricciones definidas en el ordenador central o procesador obligan a limitaciones en la aplicación.	4
Además de las características del punto 4, existen limitaciones en los componentes distribuidos del sistema.	5

Limitaciones de procesador o memoria que existen en las máquinas en que se usará el software.

Paso 4: Análisis de las características generales del sistema

Frecuencia de Transacciones	
No existe una definición del periodo punta de transacciones.	0
Se conoce el periodo punta (mensual, trimestral, etc).	1
Se conoce el periodo punta mensual.	2
Se conoce el periodo punta diario.	3
La frecuencia de transacciones definida por el usuario en los requisitos de la aplicación son suficientemente altos como para requerir un análisis específico de los rendimientos de tareas durante la fase de diseño.	4
La frecuencia de transacciones definida por el usuario en los requisitos de la aplicación son suficientemente altos como para requerir el uso de herramientas de análisis de los rendimientos de tareas y de herramientas de medida del rendimiento en el diseño, codificación y/o instalación.	5

Paso 4: Análisis de las características generales del sistema

Entrada de datos on-line	
Todas las entradas se realizan por lotes.	0
Del 1% al 7% de las entradas son interactivas.	1
Del 8% al 15% de las entradas son interactivas.	2
Del 16% al 23% de las entradas son interactivas.	3
Del 24% al 30% de las entradas son interactivas.	4
Más del 30% de las entradas son interactivas.	5

Paso 4: Análisis de las características generales del sistema

Eficiencia del usuario final.

Funciones:

- Ayudas a la navegación.

- Menús.

- Ayudas on-line.

- Scrolling.

- Impresión remota.

- Teclas de función preasignadas.

- Selección mediante cursor de datos en pantalla.

- Uso de ratón. Ventanas.

- ...

Paso 4: Análisis de las características generales del sistema

Eficiencia del usuario final	
Ninguna función.	0
1-3 funciones.	1
4-5 funciones.	2
6 o más funciones, pero no existen requisitos de usuario respecto a la eficiencia.	3
6 o más funciones, pero están definidos los requisitos de eficiencia del usuario que obligan a diseñar tareas que tienen en cuenta factores humanos (p.ej. minimizar el número de tecleos).	4
6 o más funciones, y hay requisitos del usuario sobre eficiencia que obligan a utilizar herramientas especiales y procesos para demostrar que los objetivos se han cumplido.	5

Paso 4: Análisis de las características generales del sistema

Actualizaciones on-line.	
Ninguna.	0
Actualización on-line de 1 a 3 ficheros.	1
Actualización on-line de 4 o más ficheros.	2
Actualizaciones importantes de los Ficheros Lógicos Internos.	3
Además del punto 3, la protección contra la pérdida de datos es esencial y ha sido especialmente diseñada y programada en el sistema.	4
Además del punto 4, existe un alto volumen de transacciones y los procedimientos de recuperación están altamente automatizados con intervención mínima del usuario.	5

Paso 4: Análisis de las características generales del sistema

Procesos complejos	
Ninguna de las siguientes categorías: ■ Controles especiales (auditorías) y/o aplicaciones de seguridad ■ Procesos lógicos complejos. ■ Procesos matemáticos complejos. ■ Excesivas excepciones de proceso. ■ Manejo de dispositivos complejos (multimedia, etc).	0
Una de las categorías anteriores.	1
Dos de las categorías anteriores.	2
Tres de las categorías anteriores.	3
Cuatro de las categorías anteriores.	4
Todas las categorías anteriores.	5

Paso 4: Análisis de las características generales del sistema

Reusabilidad	
No reusable	0
Se utiliza código reusable dentro de la aplicación.	1
Sobre el punto 1, menos del 10% de la aplicación tiene en cuenta las necesidades de más de un usuario.	2
Sobre el punto 1, el 10% o más de la aplicación tiene en cuenta las necesidades de más de un usuario.	3
La aplicación fue expresamente realizada y documentada para ser fácilmente reusable. La aplicación es adaptada por el desarrollador a nivel de código fuente.	4
La aplicación fue expresamente realizada y documentada para ser fácilmente reusable. La aplicación es adaptada por el desarrollador o el usuario por medio de parámetros de mantenimiento	5

La aplicación (código y documentación) ha sido específicamente diseñada e implementada para ser reutilizada (toda o parte) en otras aplicaciones.

Mgter. Oscar Adolfo Vallejos – FaCENA - UNNE

Paso 4: Análisis de las características generales del sistema

Facilidad de instalación	
No existen requisitos especiales de instalación ni se requieren desarrollos especiales.	0
No existen requisitos especiales de instalación pero sí se requieren desarrollos especiales para la instalación.	1
Los requisitos de instalación fueron definidos por el usuario.	2
Los requisitos de instalación fueron definidos por el usuario y las guías de conversión e instalación fueron proporcionadas y probadas.	3
Además del punto 2, se proporcionaron y probaron herramientas para la instalación.	4
Además del punto 2, se proporcionaron y probaron herramientas para la instalación y revisión automática.	5

Requisitos de los usuarios respecto a la facilidad de instalación

Paso 4: Análisis de las características generales del sistema

Facilidad de operación	
No se definieron por parte del usuario necesidades especiales de operación.	0
Seleccionar, valorando como uno, cada una de las siguientes solicitudes realizadas a la aplicación: ■ Procesos eficaces de arranque y recuperación (contar como 2). ■ La aplicación minimiza la necesidad de uso/manejo de cintas. ■ La aplicación minimiza la necesidad de uso/manejo de papel.	1-4
La aplicación debe diseñarse para que los usuarios sólo tengan que intervenir en el proceso de arranque y parada de la aplicación.	5

Requisitos o necesidades de operación, arranque eficaz, procedimientos de respaldo y recuperación.

Paso 4: Análisis de las características generales del sistema

Instalación en lugares distintos	
No existen requisitos de usuario para considerar la necesidad de más de un usuario o lugar de instalación.	0
Se necesita diseñar la aplicación para ser utilizada en múltiples lugares pero funcionará bajo entornos idénticos de hardware y software.	1
Se necesita diseñar la aplicación para ser utilizada en múltiples lugares pero funcionará bajo entornos similares de hardware y software.	2
Se necesita diseñar la aplicación para ser utilizada en múltiples lugares pero funcionará bajo entornos distintos de hardware y software.	3
Deberán ser proporcionados y probados la documentación y los planes de soporte de la aplicación para ser utilizados en distintos lugares en el modo que se indicó en los puntos 1 ó 2.	4
Deberán ser proporcionados y probados la documentación y los planes de soporte de la aplicación para ser utilizados en distintos lugares en el modo que se indicó en el punto 3.	5

Paso 4: Análisis de las características generales del sistema

Facilidad de cambio	
No existe ningún requisito por parte de los usuarios para facilitar el cambio (ej. distintas entradas al sistema, flexibilidad en las consultas o salidas, etc.)	0
Se seleccionarán alguna de estas opciones: ■ Facilidad para realizar consultas o informes simples tales como la utilización de operadores lógicos AND/OR sobre un fichero lógico interno. (Contar como 1). ■ Facilidad para realizar consultas o informes de complejidad media tales como la utilización de operadores lógicos AND/OR sobre más de un fichero lógico interno. (Contar como 2). ■ Facilidad para realizar consultas o informes complejos. (Contar como 3). ■ Se mantendrán datos de control en tablas que serán mantenidos por los usuarios a través de procesos interactivos on-line, pero los cambios no serán efectivos inmediatamente (Contar como 1). ■ Igual que en el caso anterior, pero los cambios serán efectivos inmediatamente. (Contar como 2).	1-5

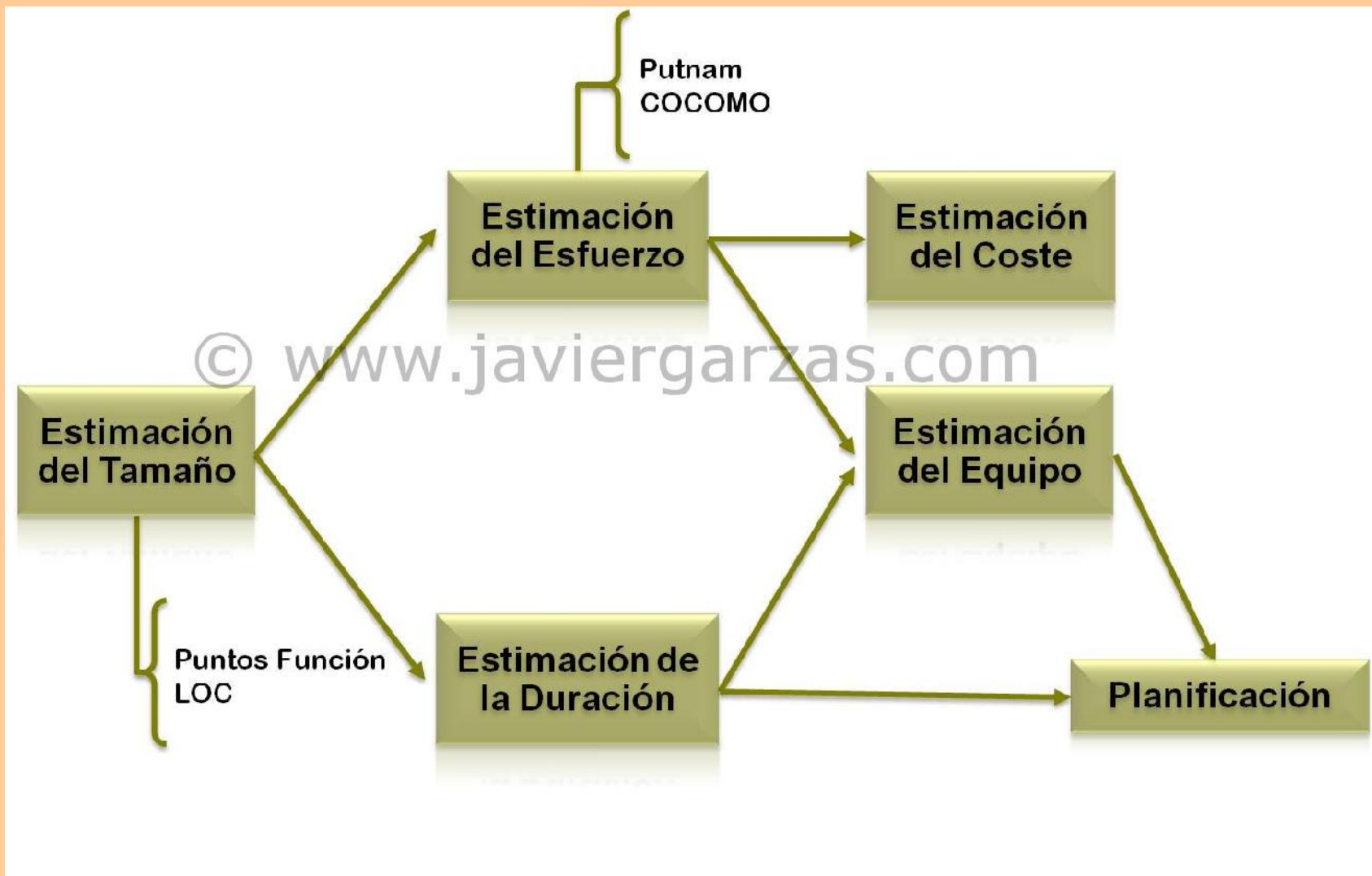
Puntos de Función: Ratios

Los puntos de función permiten normalizar medidas de productividad, calidad y otros ámbitos del software:

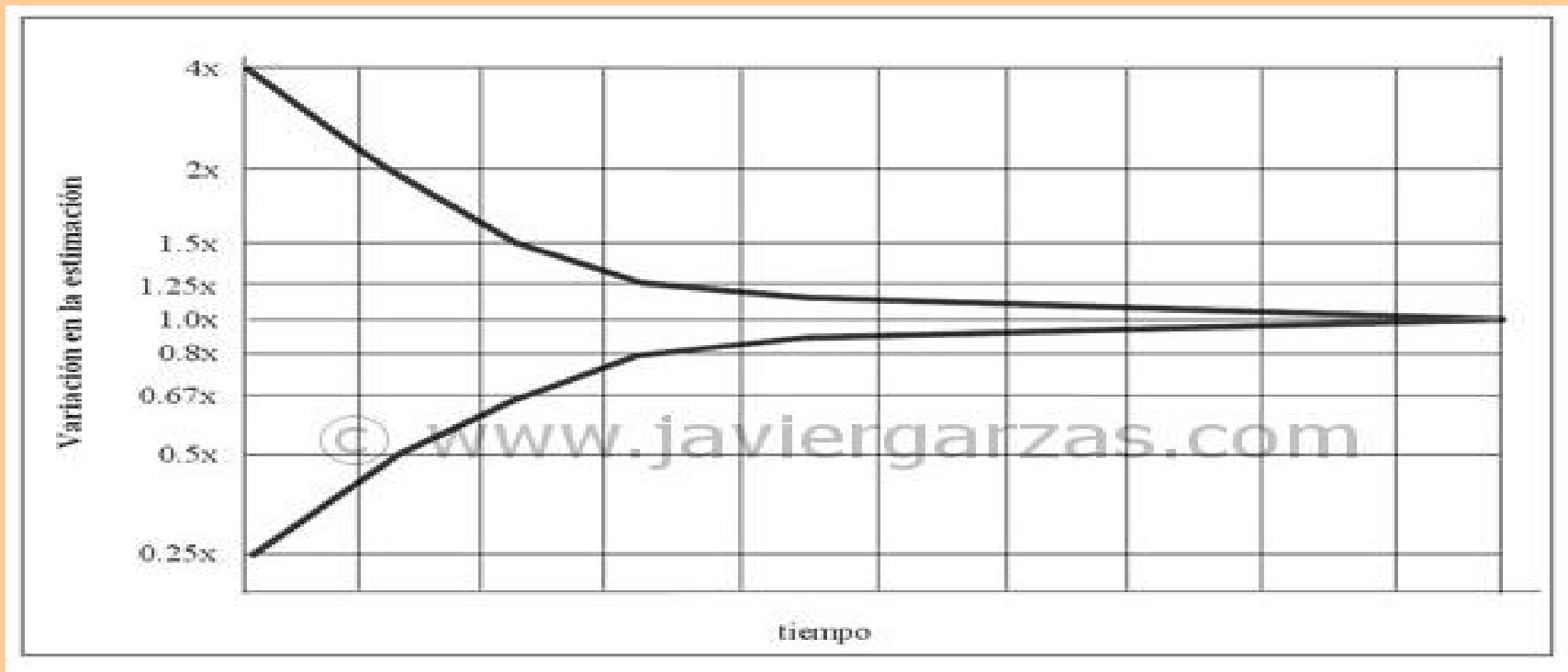
- ❑ Productividad = puntos de función/persona-mes.
- ❑ Calidad = errores/punto de función.
- ❑ Coste = Euros/punto de función.
- ❑ Documentación = páginas/punto de función.
- ❑ Líneas de código = líneas/punto de función.

Puntos de Función: Ratios

Lenguaje	Media de instrucciones fuente por punto de función
Ensamblador	320
C	128
FORTRAN 77	105
ANSI COBOL 85	91
Pascal	91
PL/I	80
Ada	71
QuickBasic	74
Lenguaje estadístico	32
Lenguaje Orientado a Objetos	29
4GL	20
Lenguaje de consulta	13



- **Estimar varias veces durante el proyecto.** No basta con estimar al principio, según avanza el proyecto deberíamos reajustar la estimación. Algunos autores señalan que deberíamos estimar al menos en tres puntos: En la etapa de estudio de viabilidad, o inicio del proyecto, en la etapa de requisitos y en la etapa de diseño. Yo incluso creo que en proyectos grandes habría que estimar varias veces según avanza el desarrollo.
- **El nivel de precisión de la estimación es distinto a medida que avanza el proyecto.** Al principio del proyecto, cuando sólo se tienen requisitos, el error con el que se trabaja es mayor que cuando se estima en la fase de diseño. Una figura muy útil para entender esto es el “cono de incertidumbre”, que muestra cómo las estimaciones son más precisas según progresa el proyecto:



Aparte de tamaño y esfuerzo global, hay que estimar fases concretas del ciclo de vida. Si no se tienen históricos, existen fórmulas que dicen cómo se divide en fases el esfuerzo total del proyecto, las más famosas son del ISBSG. **Cada empresa (y muchas veces tipo de proyecto) tiene su método de estimación.** No hay un mejor método de estimación... hay uno mejor para cada empresa, y este depende del negocio, del tamaño de la empresa, de su madurez, etc. Por ejemplo, el método de estimación para empresas que desarrollan productos no será seguramente igual que el de empresas que externalizan.

Utilizar varias fuentes y métodos. Complementar siempre el método de estimación con datos de proyectos anteriores (analogía), y realizar comparativas con los mismos. Además, complementar preguntando al equipo de desarrollo.

Usa una unidad de estimación acorde a la fase del proyecto en la que estimas. A la hora de proporcionar una estimación muchas veces es útil valerse de rangos, que se pueden ir ajustando conforme avanza el proyecto. Por ejemplo, *“entre la primera y la segunda semana del mes de abril”* o *“10 meses con más menos quince días de error”*. Un error muy frecuente es en la primera estimación decir el día exacto de finalización del proyecto, cuando sabemos que en una estimación tan temprana siempre hay error.

Técnicas de estimación: Herramientas automáticas

- ❑ Implementan de forma automática técnicas de descomposición o modelos empíricos.
- ❑ Permiten estimar costes, esfuerzo, riesgos, productividad, personal medio por mes, etc...
- ❑ Análisis de diversos escenarios (“¿Qué pasaría si...?”).
- ❑ *Se requiere experiencia para su uso.*

Técnicas de estimación: Herramientas automáticas

Datos de entrada:

Estimación del tamaño del proyecto (p.e. LDC) o funcionalidad (puntos de función).

Características cualitativas del proyecto (complejidad, fiabilidad requerida, grado de criticidad, etc.)

Descripción del personal y entorno de desarrollo.

Permiten simular diversos escenarios.

Herramientas automáticas - CoStar

CoStar - Estimate1

File View Reports Constraints Preferences Help

Estimate Name: Estimate1 ID:

Totals for entire Project		Effort (PM)	Duration (Mo)	Cost (K \$)	Total Size: 0
Requirements	RQ:	0.0	0.0	0.0	
Development	PD+DD+CT+IT:	0.0	0.0	0.0	
Total	RQ+PD+DD+CT+IT:	0.0	0.0	0.0	

COCOMO Model Name:
COCOMO 87

Model Type: Traditional
Model ID: 1987
Phases: Waterfall

Select COCOMO Model

COCOMO Calculation Mode

Development Mode: Semidetached

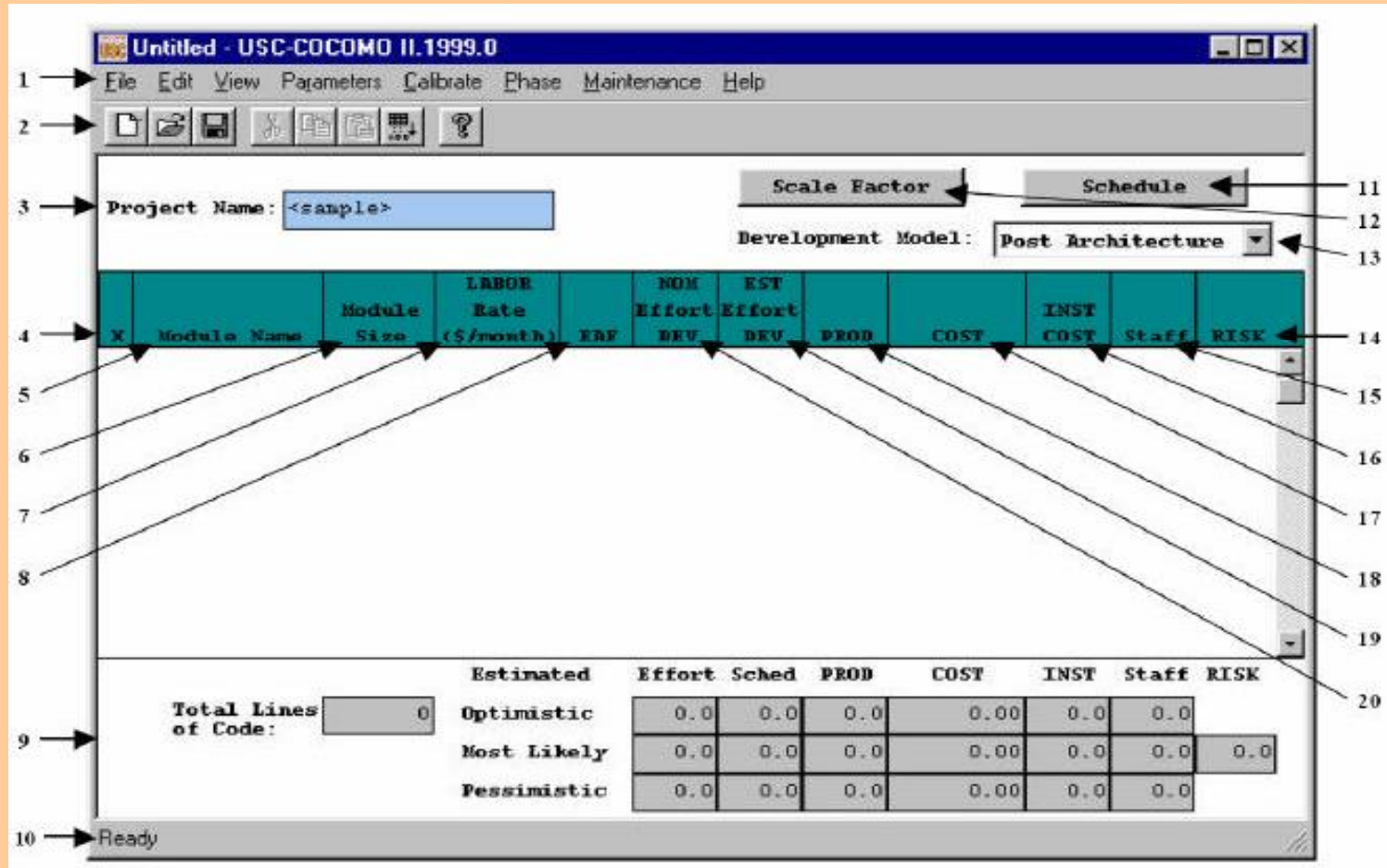
Calculation Type: Intermediate

Model Equations Increment Phasing Increment Breakage Labor Cost APM MN APM Description

<http://www.softstarsystems.com/demo.htm>

Mgter. Oscar Adolfo Vallejos – FaCENA - UNNE

ESTIMACIÓN DE ESFUERZO CON USC COCOMO II



Costes

Dependen de:

- ☐ Alcance del proyecto.
- ☐ Duración.
- ☐ Estimación de recursos hardware.
- ☐ Estimación de recursos software.
- ☐ Estimación de recursos humanos.

Coste del personal

- Establecer un coste unitario Categoría/Unidad de esfuerzo (persona/día, persona/mes, persona/año) llamado *coeficiente de trabajo*.
- *Calcular el esfuerzo estimado para cada categoría.*
- *Calcular el coste de cada categoría.*

ci = Coeficiente de trabajo x Esfuerzo estimado

- *Calcular el coste total:*

$$c = \sum c_i$$

Costes globales



Planificación. Elementos a planificar

- ☐ Alcance del sistema software.
- ☐ Proceso de desarrollo:
 - ☐ Ciclo de vida.
 - ☐ Fases.
 - ☐ Técnicas y métodos.
 - ☐ Software/Hardware.
 - ☐ Hitos.
 - ☐ Resultados.
- ☐ Desglose en tareas y relaciones entre ellas.
- ☐ Recursos y distribución.
- ☐ Tiempos.
- ☐ Formación del equipo de desarrollo.
- ☐ Garantía de calidad.
- ☐ Procedimientos de evaluación y seguimiento.
- ☐ Gestión de configuraciones.
- ☐ Análisis del riesgo.

Definir restricciones bajo las que se debe llevar a cabo el proyecto
Realizar juicios iniciales sobre los parámetros del proyecto
Definir hitos y entregas

Mientras (proyecto no completado o cancelado) **hacer**

Trazar el calendario del proyecto

Iniciar actividades de acuerdo al calendario

Esperar un tiempo

Revisar el progreso del proyecto

Aplicar revisiones al calendario del proyecto

Renegociar restricciones del proyecto y entregas

Si (hay problemas) **entonces**

Iniciar examen técnico y posible revisión

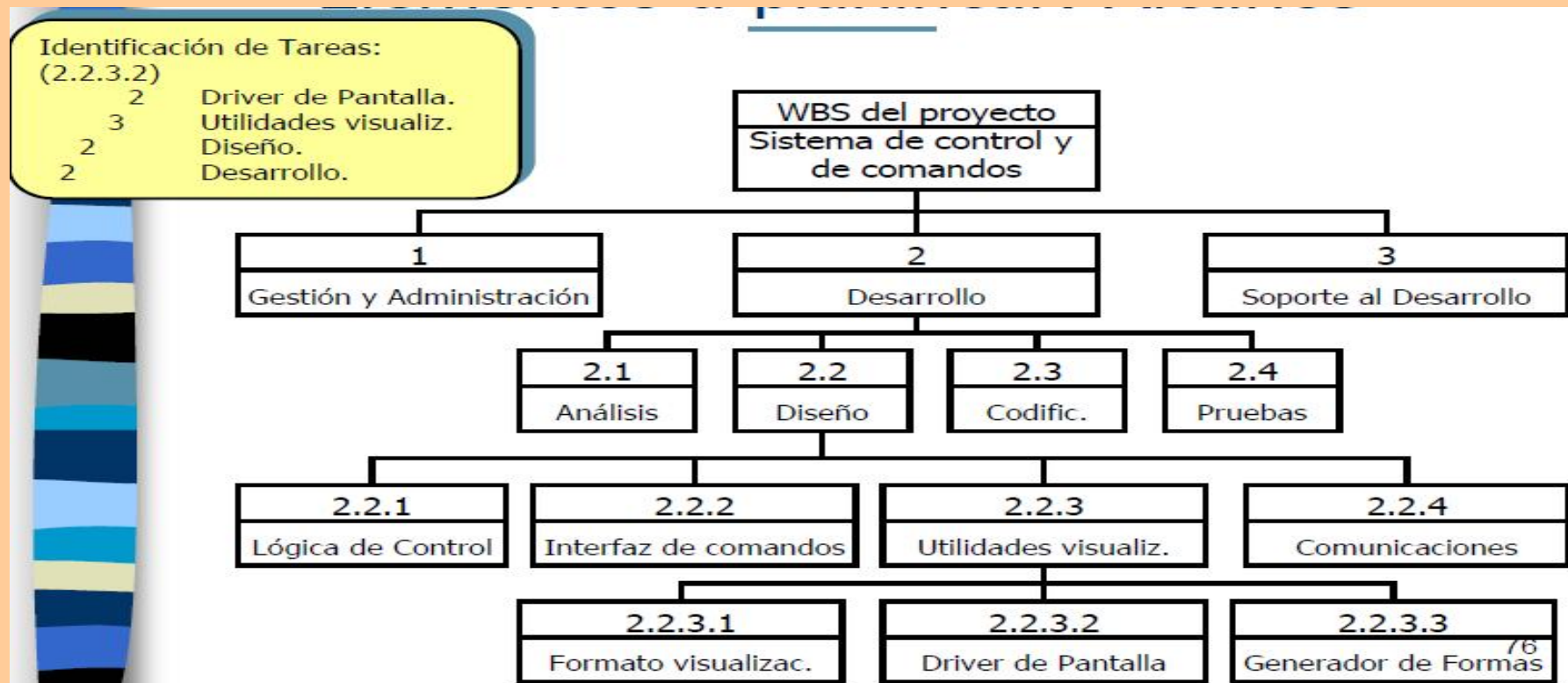
Fin-Si

Fin-Mientras



Elementos a planificar: Alcance

- ❑ Qué va a ser parte de la aplicación y qué no.
- ❑ Tareas a realizar/productos a generar.
- ❑ Work breakdown structure (WBS).
- ❑ *Dividir un proyecto en partes más pequeñas, hasta llegar al nivel fundamental: unidades de trabajo (work packages).*
- ❑ *Orientado al producto vs. Orientado al proceso.*



Planificación. Recursos

- ☐ Consideraciones a tener en cuenta para planificar...
 - ☐ La duración de cada tarea.
 - ☐ La fecha de inicio y de finalización.
 - ☐ Los recursos a asignar a cada tarea.
- ☐ La planificación depende de:
 - ☐ Complejidad de la tarea.
 - ☐ Modificaciones permitidas a lo largo del desarrollo.
 - ☐ Experiencia previa del equipo de trabajo.
 - ☐ Duración fijada del proyecto.
 - ☐ Estructuración del problema y las tareas.
 - ☐ Disponibilidad de datos e información suministrada por el usuario.
 - ☐ Disponibilidad y facilidad de comunicación con el usuario.

Planificación. Recursos

Recursos humanos.

Perfil requerido (experiencia, conocimientos, etc.)

Disponibilidad.

Duración. Fecha de comienzo.

Dedicación.

Recursos hardware/Software.

Descripción.

Disponibilidad (precio).

Duración.

Fecha de compra/uso.



Planificación. Recursos

Recursos Software.

Decisión sobre:

- Adquirir un paquete o herramienta software y utilizarlo tal cual.

- Adquirir un paquete y modificarlo.

- Construirlo.

Pasos:

- Especificar la funcionalidad y rendimientos deseados.

- Estimar el coste de desarrollo interno.

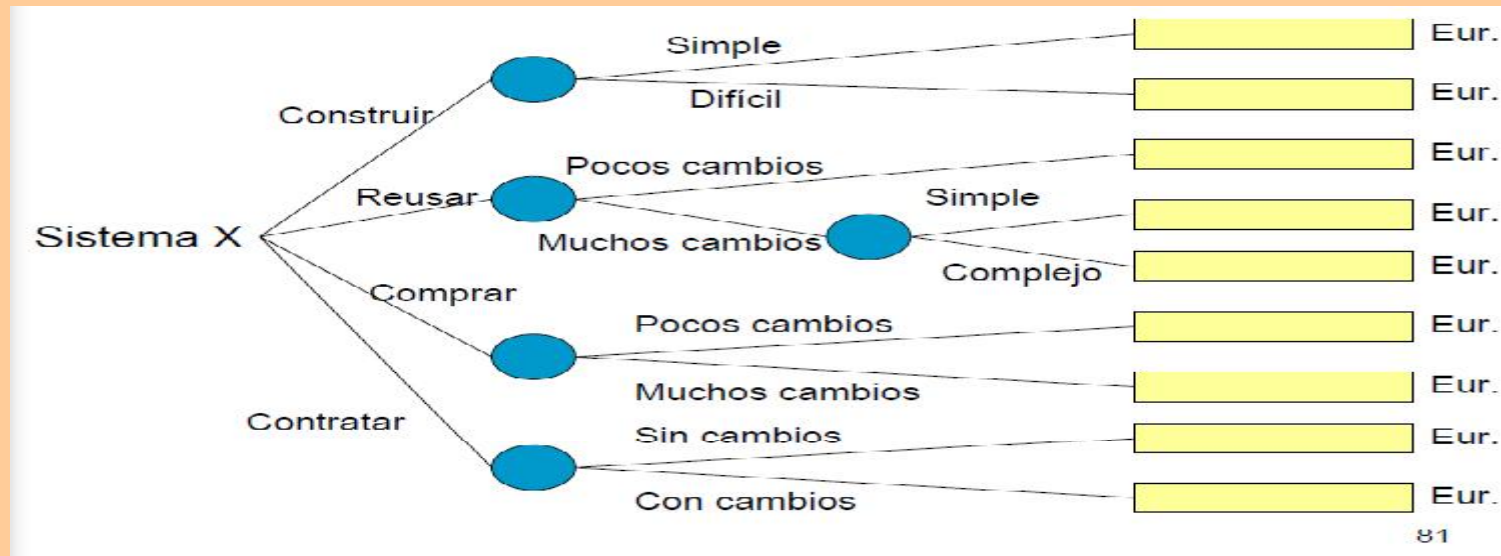
- Estimar si hay recursos humanos disponibles para que esté acabado en el tiempo previsto.

- Seleccionar paquetes comerciales.

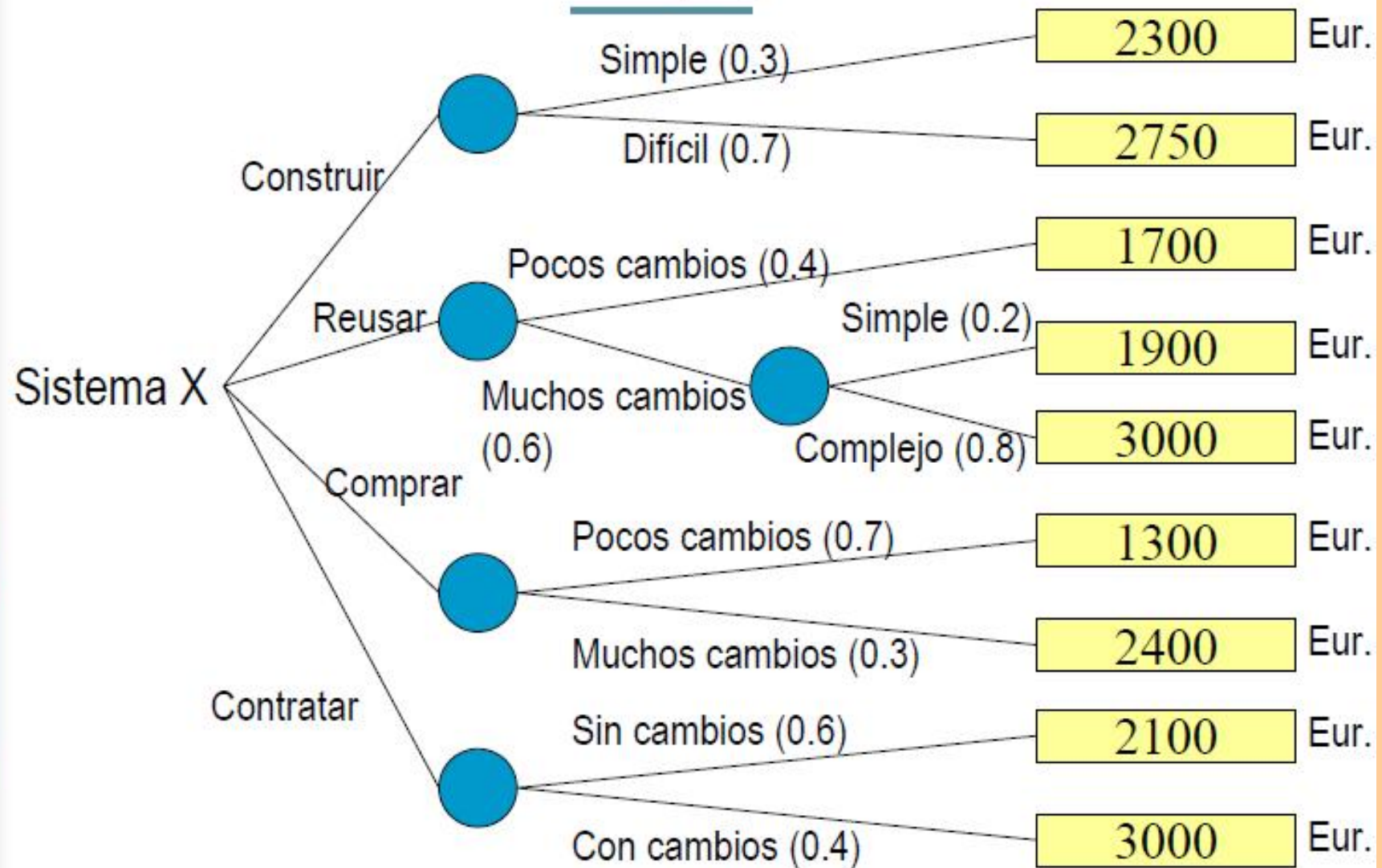
- Evaluarlos (precio, características, soporte, calidad,...).

- Decidir

Planificación. Recursos. Arbol de decisión



- ❑ Estimación de costes usando las técnicas estudiadas.
- ❑ Probabilidades en los arcos.
- ❑ Coste esperado= $\Sigma(\text{probabilidad del camino}) \times (\text{coste estimado del camino})$.



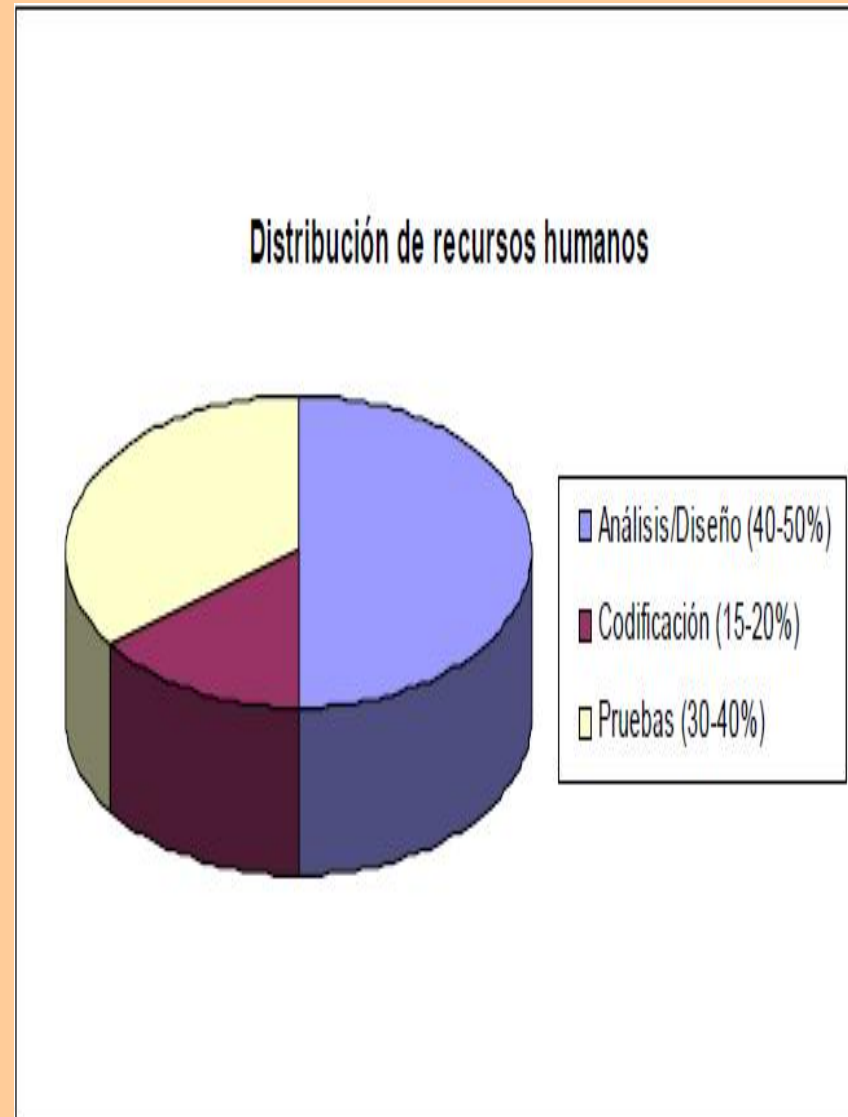
■ $CE_{cons} = 0.3 \times 2300 + 0.7 \times 2750 = 2615 \text{ Euros.}$

Planificación. Distribución de recursos

Depende del tipo de proyecto:

En SW de tiempo real las pruebas pueden llegar al 50%.

En SW de gestión las fases de análisis y diseño son más costosas.



Planificación Temporal. Técnicas de Ayuda

Técnica de revisión y evaluación del programa(*PERT*).

Entradas:

Tareas y relaciones.

Duración total de las tareas.

Esfuerzo dedicado a cada tarea.

Lo más pronto y lo más tarde que pueden empezar/acabar las tareas.

CPM, Salidas:

Camino crítico.

Fechas de inicio/fin más tempranas para cada tarea.

Fechas de inicio/fin más tardías para cada tarea.

Holgura para cada tarea.

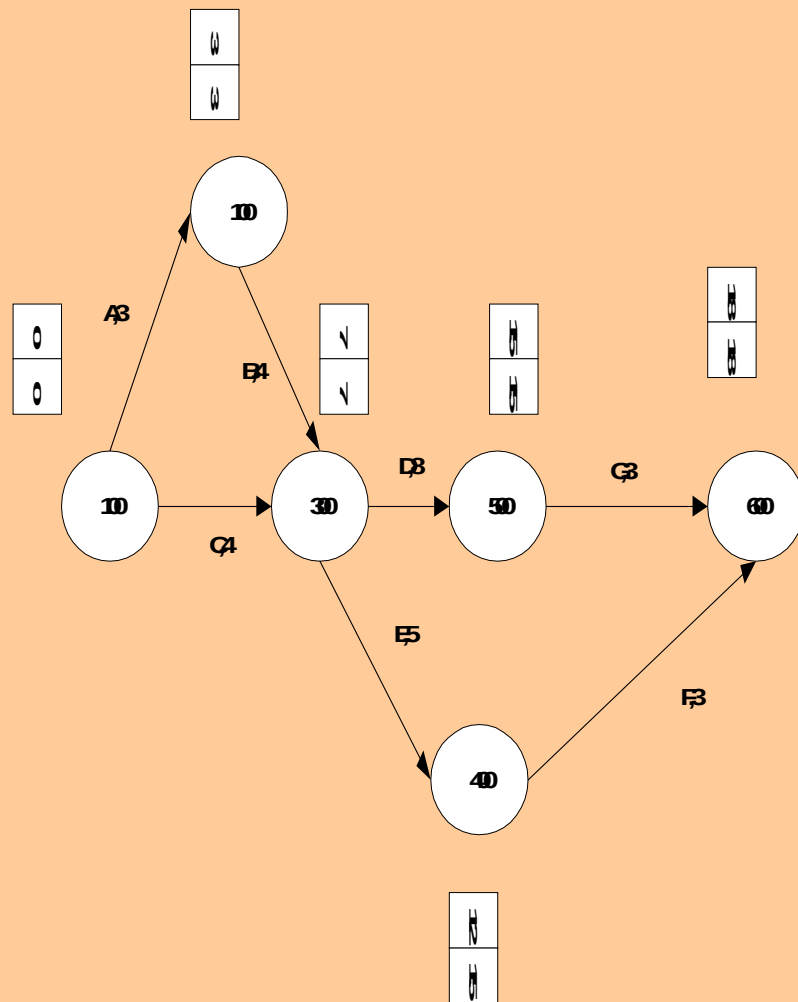
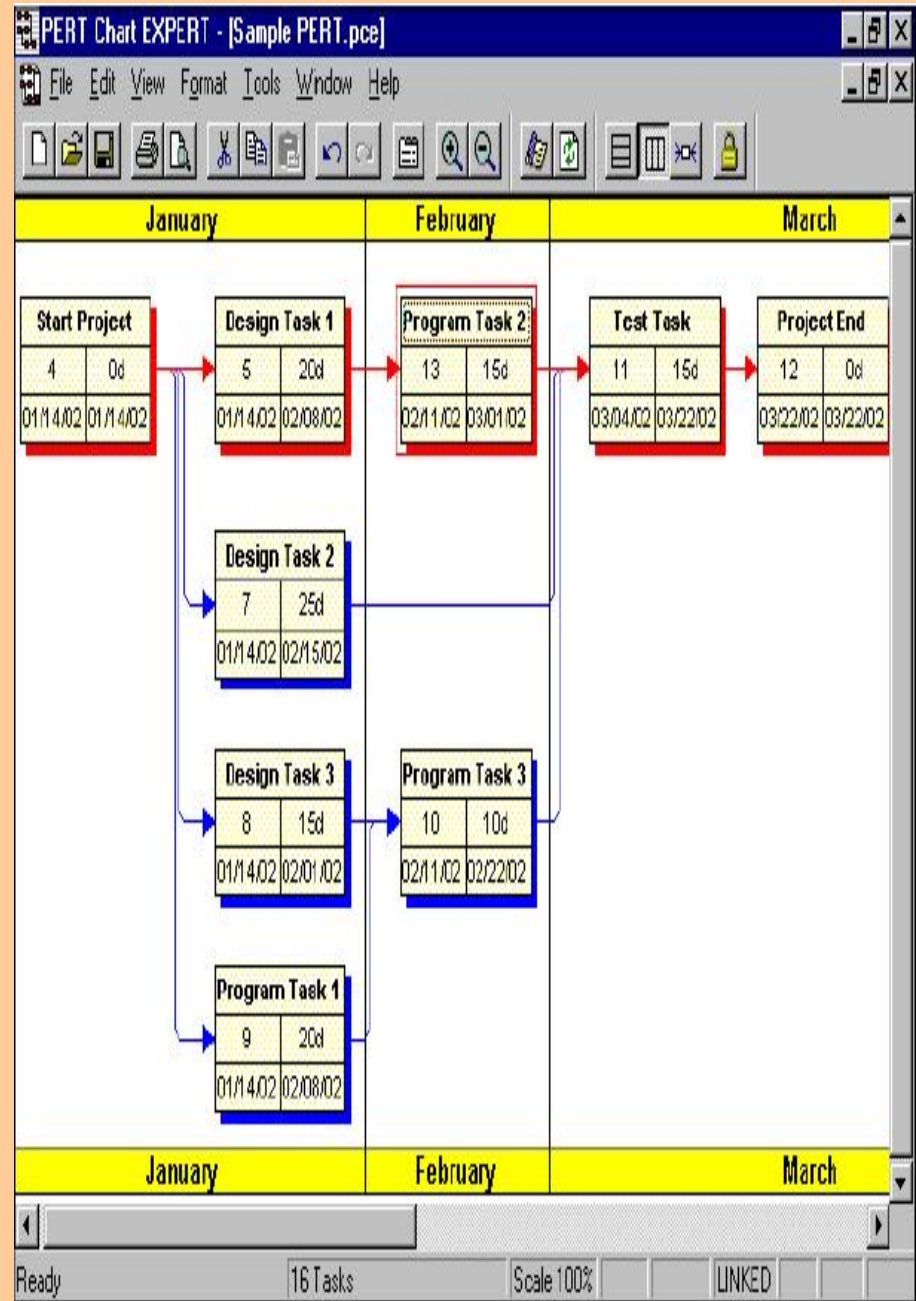


Fig 0211 Diagram PERT



Planeación de actividades: PERT

PERT (Program Evaluation and Review Techniques).

Se representan por una red de flechas y nodos, que luego se evalúa para determinar cuales son las actividades criticas y mejorar su programación, si fuese necesario, como también para revisar el avance del proyecto.

Las actividades se representan por la flechas, la longitud de la flecha no esta asociado a la duración de la actividad. Los nodos circulares, también llamados eventos, sirven para:

- reconocer que una actividad ha concluido
- indicar que actividades necesitan concluirse antes de iniciar una nueva (precedencia)

Los nodos, para su identificación, pueden tener número, letras u otra cualquier identificación.

Planeación de actividades: PERT

A la trayectoria mas larga se le denomina ruta critica, es el trayecto más largo y un atraso en dicho trayecto implicara un retraso en todo el proyecto.

Para mantener la lógica y caridad de los diagramas es necesario la utilización de actividades ficticia (duración cero) o dummy.

El tiempo de holgura puede obtenerse para cada uno de los eventos y con ello se sabrá el del tiempo que se dispone para concluir una actividad antes de que se arriesgue la duración global de proyecto.

Para obtener la holgura, primero deberá determinarse el **fin temprano (FT)** y el **inicio tardío (IT)**.

El fin temprano de un evento es el momento en el que más tempranamente es posible concluir todas las actividades que convergen a tal nodo.

El inicio tardío es el ultimo momento disponible para iniciar aquellas actividades que se originan en tal nodo. La diferencia entre el FT y IT es la holgura particular del evento.

El diagrama de pert nos permite:

- la identificación fácil del orden de la precedencia
- la identificación fácil de la ruta critica; y consecuentemente, de las actividades criticas
- el calculo sencillo de la duración de la holgura

Planificación Temporal. Técnicas de Ayuda

☐ Diagramas de GANTT

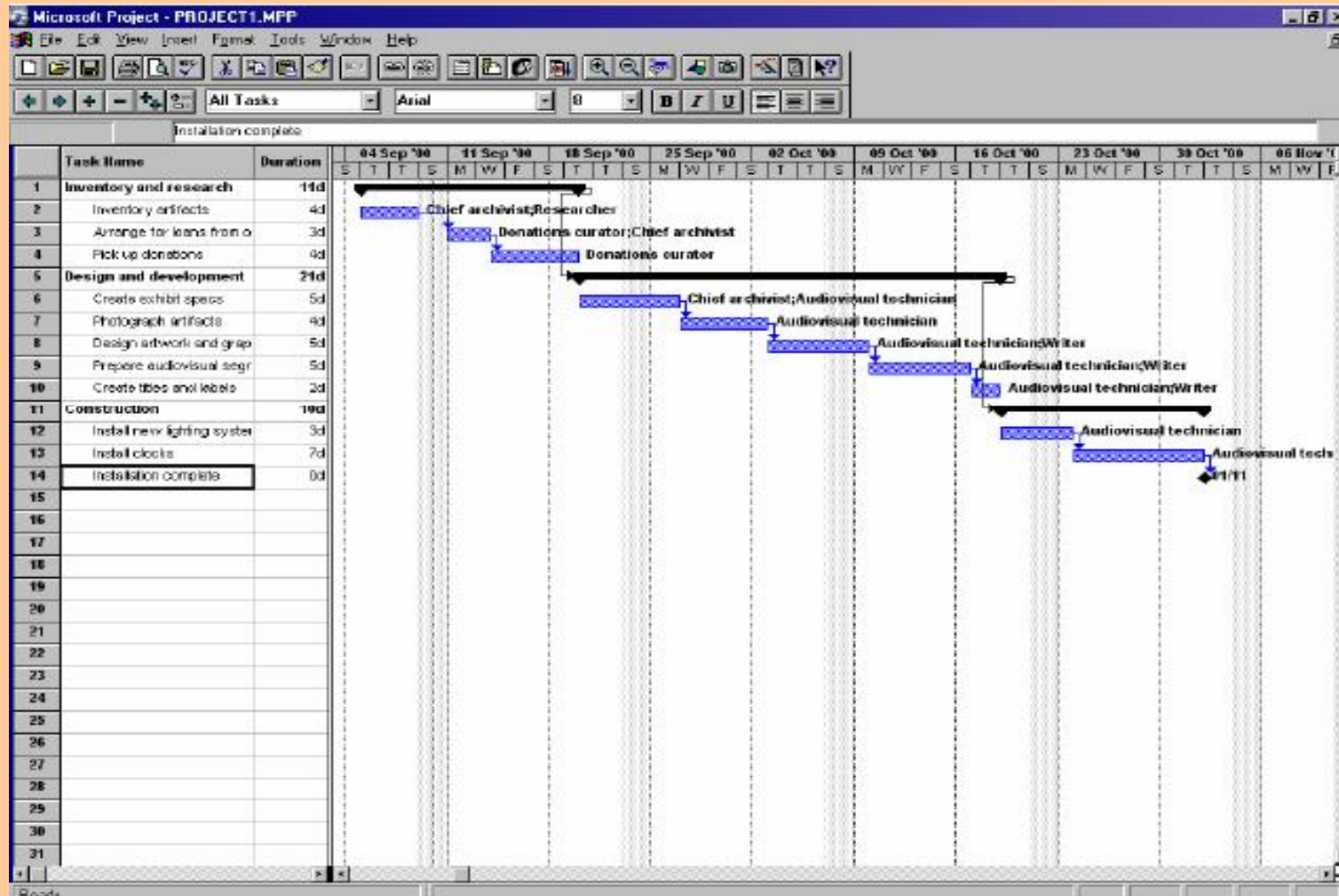
☐ Entradas:

- ☐ Tareas.
- ☐ Duración de las tareas.
- ☐ Fechas de inicio y fin de las tareas.

☐ Salidas:

- ☐ Presentación de la planificación temporal de las tareas.

Diagramas de GANTT



Planeación de actividades: GANTT

Esencialmente es un diagrama que contiene barras que representan cada una de las actividades, y cuya longitud representa la duración de la actividad respectiva.

Un diagrama de gantt de una dimensión es un calendario, siendo una técnica muy usada para la planeación de las actividades.

Cuando las actividades se realizan en forma simultanea la resultante es un diagrama de dos dimensiones

Planificación. Creación del equipo de desarrollo

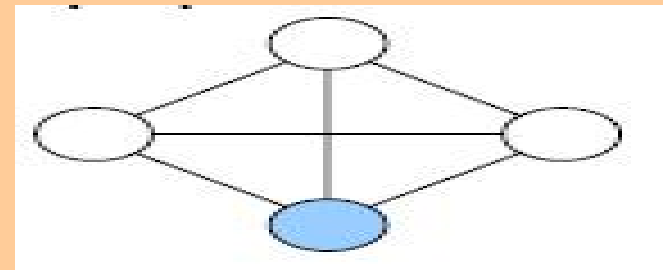
- Definir estructura y organización.
- Revisar miembros del equipo y papeles.
- Definir canales de comunicación.



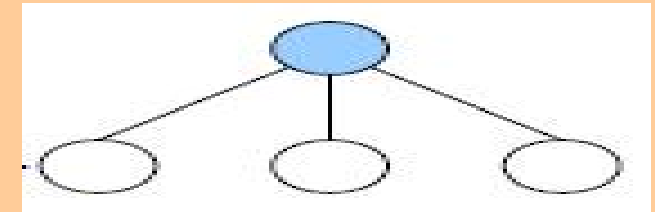
Planificación. Creación del equipo de desarrollo

Estructura:

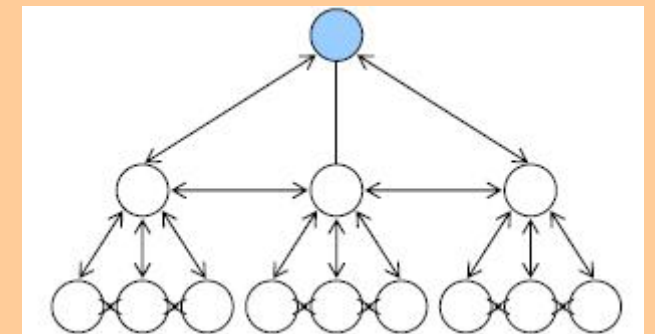
☐ Democrático.



☐ Basado en Jefes de Proyecto.



☐ Jerárquico.



Planificación. Creación del equipo de desarrollo

Cada miembro del equipo de desarrollo tiene que tener claro:

- ¿ Qué se espera de mi ?.
- ¿ Cómo puedo realizar eso que se espera de mi ?.
- ¿ Qué debo producir ?.
- ¿ Cómo se va a evaluar mi producción ?.
- ¿ Cuáles son mis tareas específicas y de cuánto tiempo dispongo?.
- ¿ Qué herramientas/formación tengo disponible ?.
- ¿ Cómo está organizado el equipo y cuáles son los caminos de comunicación ?.
- ¿ A quién tengo que recurrir si tengo problemas ?.

Sentirse importante:

- Hacerle importante.
- Darle “responsabilidad” en lo que haga.
- Información (reuniones periódicas).

Planificación. Creación del equipo de desarrollo

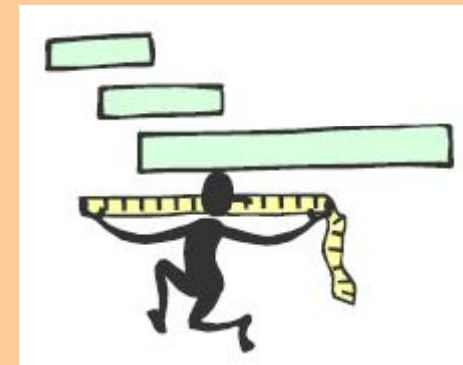
Etapas por las que pasa el equipo de desarrollo:

- ☐ Formación.
- ☐ Discrepancias.
- ☐ Normalización.
- ☐ Eficacia.
- ☐ Fin.



Planificación. Otras actividades

- ☐ Revisión de las identificaciones y definiciones realizadas previas al desarrollo.
- ☐ Mecanismos de seguimiento y control:
 - ☐ Cumplimiento de tiempos.
 - ☐ Evaluación de objetivos.
 - ☐ Coordinación del equipo de desarrollo.
 - ☐ Revisiones



Planificación. Perfil del planificador

- ❑ Gran formación y experiencia profesional.
- ❑ Posicionen la organización que le permita tomar decisiones y adoptar un juicio independiente.
- ❑ Visión global del proyecto y de la organización.

Planificación. Errores típicos

- ☐ Optimismo en la estimación.
- ☐ No tener en cuenta tiempos adicionales al desarrollo.
- ☐ No usar el plan. No realizar un seguimiento del proyecto.
- ☐ No actualizar el plan.
- ☐ No haber identificado riesgos.
- ☐ No repartir las responsabilidades adecuadamente.
- ☐ No haber definido criterios de evaluación.
- ☐ Añadir más personas cuando el proyecto va retrasado.

Plan de proyecto

❑ Establece el **Qué** e informa del **Cuánto** y **Cómo** de largo.

❑ Debe:

❑ Determinar el alcance del proyecto.

❑ Especificar los recursos.

❑ Definir el coste.

❑ Detallar el plan temporal.

Plan de proyecto

1. Alcance.

- Objetivos del proyecto.
- Funciones principales.
- Otras características.
- Escenario de desarrollo.
- Restricciones técnicas y de gestión.

2. Estimación de recursos.

- Recursos humanos.
- Recursos hardware y software.
- Ventanas de disponibilidad.

3. Estimación de coste.

4. Plan temporal.

- Red de tareas.
- Diagrama de Gantt.
- Tabla de tareas/recursos.

5. Equipo de desarrollo.

6. Requisitos de disponibilidad del cliente.

- Información/Datos.
- Reuniones.
- Otros

Análisis de Riesgos

Riesgo

- Es la probabilidad de que una circunstancia adversa ocurra.
- Los riesgos son una amenaza para el proyecto, para el software que se esta desarrollando y para la organización.

Análisis de Riesgos

- **Riesgo del proyecto:** afecta la calendarización o los recursos del proyecto.
- **Riesgo del producto:** afecta la calidad o rendimiento del software.
- **Riesgo del negocio:** afecta a la organización que desarrolla o suministra el software.

Riesgo	Tipo	Descripción
Rotación del personal	Proyecto	Personal con experiencia abandona el proyecto.
Competencia del producto	Negocio	La tecnología fundamental sobre la que se constituirá el sistema se sustituye por nueva tecnología.
Cambio de requerimientos	Producto	Habrà mas cambios de lo esperado

Análisis de Riesgos

El proceso de gestión de riesgos, comprende varias etapas:

- **1. Identificación:** identificar los posibles riesgos para el proyecto, el negocio y el producto. (Se puede llevar a cabo a través de una tormenta de ideas o basarse en la experiencia)
- **2. Análisis:** valorar las probabilidades y consecuencias de estos riesgos.
- **3. Planificación:** crear planes para abordar los riesgos, para evitarlos o minimizar sus efectos.
- **4. Supervisión:** valorar los riesgos de forma constante y revisar los planes.

Análisis de Riesgos

1.- Identificación

Hay al menos 6 tipos de riesgos:

- **Riesgos de tecnología:** se derivan de las tecnologías de software o de hardware utilizados.
- **Riesgos de personal:** riesgo asociado con las personas del equipo de desarrollo.
- **Riesgos organizacionales:** se derivan del entorno organizacional donde se desarrolla el software.
- **Riesgo de herramientas:** se derivan de herramientas CASE y de otro software de apoyo utilizado en el desarrollo.
- **Riesgos de requerimientos:** se derivan de los cambios de los requerimientos del cliente y el proceso de gestionar dicho cambio.
- **Riesgos de estimación:** se derivan de los estimados administrativos y los recursos requeridos.

Análisis de Riesgos

Riesgo	Tipo
Los componentes de software que deben reutilizarse contienen defectos.	Tecnología
La capacitación solicitada para el personal no esta disponible	Personal
La organización se reestructura de tal forma que una gestión diferente se responsabiliza del proyecto	Organizacional
Es ineficiente el código generado por las herramientas CASE	Herramientas
Se proponen cambios que requieren rehacer el diseño	Requerimientos
El tiempo requerido para desarrollar el software esta subestimado	Estimación

Análisis de Riesgos

2. Análisis

- Se considera por separado cada riesgo identificado y se decide acerca de la probabilidad y la seriedad del mismo.
- Se realiza en base a la opinión y experiencia del gestor del proyecto.
- No se hace una valoración con números sino en intervalos:

Probabilidad de Ocurrencia	Nivel de Impacto
<i>Muy bajo (<10%)</i>	<i>Insignificante</i>
<i>Bajo (10% – 25%)</i>	<i>Tolerable</i>
<i>Moderado (25% - 50%)</i>	<i>Serio</i>
<i>Alto (50% - 75%)</i>	<i>Crítico</i>
<i>Muy Alto (>75%)</i>	<i>Catastrófico</i>

Análisis de Riesgos

- El resultado de este análisis se debe colocar en una tabla, la cual debe estar ordenada según la seriedad del riesgo.
- El numero exacto de riesgos a supervisar depende del proyecto. Pueden ser 5 o 15, el numero apropiado debe ser manejable.

Riesgo	Probabilidad	Efecto
Es imposible reclutar personal con habilidades requeridas para el proyecto.	Alta	catastrófico
Los componentes de software que deben reutilizarse contienen defectos.	Moderada	Serio
Es ineficiente el código generado por las herramientas CASE	Moderada	Insignificante

Análisis de Riesgos

3. Planificación

- El proceso de planificación considera cada uno de los riesgos, y las estrategias para gestionarlos.

Categorías:

- **Estrategias de prevención:** siguiendo esta estrategia, la probabilidad del riesgo se reduce.
- **Estrategias de minimización:** siguiendo esta estrategia se reducirá el impacto del riesgo.
- **Planes de contingencia:** seguir esta estrategia es estar preparado para lo peor y tener una estrategia para cada caso.

Análisis de Riesgos

Riesgo	Estrategia	Categoría
Enfermedad de personal	Reorganizar el equipo de forma que haya solapamiento en el trabajo	Minimización
Los componentes de software que deben reutilizarse contienen defectos.	Reemplazar los componentes defectuosos con los comprados de fiabilidad conocida	Prevención
Problemas financieros de la organización	Preparar un documento para el gestor principal que muestre las ventajas del proyecto	Plan de contingencia

Análisis de Riesgos

4. Supervisión

- La supervisión valora cada uno de los riesgos para decidir si es más o menos probable y si han cambiado sus efectos.
- Como no es observable en forma directa, se buscan factores para dar indicios de la probabilidad de riesgo.

Riesgo	Indicador
Tecnología	Entrega retrasada del hardware o de la ayuda del software, muchos problemas tecnológicos reportados.
Personal	Baja moral del personal, malas relaciones entre los miembros del equipo.

- La supervisión debe ser un proceso continuo y en cada revisión del progreso de gestión, cada uno de los riesgos debe ser considerado y analizado.

Bibliografia

- Plantillas de las clases de teoria. Mgter. Vallejos, Oscar
- Capitulo I de: INGENIERÍA DEL SOFTWARE. Séptima edición Ian Sommerville. ISBN: 84-7829-074-5. PEARSON ADDISON WESLEY.
- Capitulo III al VII de: INGENIERÍA DEL SOFTWARE UN ENFOQUE PRÁCTICO Quinta edición. Roger S. Pressman. MCGRAW-HILL DINTERAMERICANA DE ESPANA, S . A. U. ISBN: 0-07-709677-0